ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ РЕСПУБЛИКИ ХАКАСИЯ
«ХАКАССКИЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

ОТЧЕТ

по производственной практике
по профессиональному модулю

ПМ.01. РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

специальности 09.02.07 Информационные системы и программирование
Квалификация Программист

Студент гр. ИС(ПРО)-41 _____ Мальцева А. А.
подпись *Фамилия И.О.*

Руководитель
практики
от предприятия _____ _____ _____ Козлов А. И.
оценка *дата* *подпись* *Фамилия И.О.*

М.П.

Руководитель
практики
от ГБПОУ РХ ХПК _____ _____ _____ Черкашин Д. С.
оценка *дата* *подпись* *Фамилия И.О.*

Абакан 2026 г.

СОДЕРЖАНИЕ

1 Исследование информационной системы предприятия 3
2 Анализ предметной области 6
3 Проектирование программного модуля 8
3.1 Разработка диаграмм IDEF0 и DFD 8
3.2 Репозиторий в системе контроля версий 13
3.3 Разработка макетов интерфейса программного модуля 14
4 Разработка программного модуля 21
4.1. Разработка интерфейсов программного модуля 21
4.2. Разработка кода программного модуля 30
5 Тестирование программного модуля 38
5.1 Разработка тест-кейсов 38
5.2 Модульное тестирование 42
Приложения 46
Спецификация требований 46
Репозиторий с документами в системе контроля версий 50

1 ИССЛЕДОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ ПРЕДПРИЯТИЯ

Производственная практика проходит в государственном бюджетном учреждении здравоохранения республики Хакасия «Абаканская межрайоная клиническая больница», является крупнейшим многопрофильным медицинским учреждением Республики Хакасия, оказывающим специализированную, в том числе высокотехнологичную, медицинскую помощь населению. Больница выполняет функции межрайонного центра, обслуживая не только жителей Абакана, но и близлежащих районов.

Практика проходит в отделе информационных технологий и защиты информации под руководством начальника отдела информационных технологий и защиты информации.

Главная задача начальника отдела является организация работы информационного отделения, решая разнообразные технологические и кадровые вопросы осуществляя стратегическое планирование. Он получает запросы на цифровое обслуживание, определяет схему и приказы исполнителям, контролируя результат выполнения работы. В подчинении у начальника отдела информационных технологи и защиты информации находятся специалисты, занимающиеся следующими направлениями:

- Связист;
- Ведущий инженер программист;
- Инженер программист;
- IP телефония;
- Специалист по защите информации;
- Общие работы с сетью;
- Промед. Система.

Связисты работают с телефонией, принимают информацию с штаба, передают и записывают её. Ведущий инженер программист занимается ведением проекта или проектов по разработке программного обеспечения имеющий определенный уровень подготовки и соответствующие требования к образованию этого статуса. Несут ответственность за архитектуру, лежащую в основе разработанной программы, обучения новых разработчиков и распределения работы. Инженер программист занимается конструированием системы с элементами искусственного интеллекта, разработка чертежей и схем, разработка комплексных расчетов. Специалисты по защите информации обеспечивают конфиденциальность и сохранность данных на этапе разработки продукта, следя за защитой от внешних угроз.

Место производственной практики лучше всего подходит для разработки расширения, так как имеет сложную организационную структуру, обладает развитой IT-инфраструктурой,

использует современные системы учета и отчетности, а также имеет значительный парк медицинского оборудования, требующий систематического учета и контроля.

В организации существует всего 700 клиентских машин и 13 серверов на области прохождения практики, без учета филиалов организации. Клиентские машины и сервер работают на одной операционной системе – Astra Linux. Также характеристики машин на которых работают клиенты и сервера имеют одинаковый минимум:

- От 8Гб оперативной памяти;
- От 120Гб основной памяти HDD или SSD диска;
- Процессор Intel Pentium 7400.

Организованные сервера имеют важную роль в защите и использовании информации в организации, так, сервер персонифицированного учета лекарств по отделениям отвечает за автоматизацию процессов распределения, учета и контроля медикаментов в разных подразделениях учреждения. Он обеспечивает точность выдачи препаратов, предотвращает их нецелевое использование и формирует отчетность для фармацевтического надзора. Данные в реальном времени помогают медицинскому персоналу оперативно отслеживать наличие лекарств и минимизировать риски их нехватки.

Общий доменный сервер служит центральным узлом для управления учетными записями сотрудников, разграничения прав доступа и обеспечения сетевой безопасности. На нем хранятся политики паролей, групповые политики и настройки доступа к корпоративным ресурсам, что позволяет поддерживать стабильность работы всей ИТ-системы и защищать данные от несанкционированного доступа.

Сервер централизованного хранения медицинских изображений (PACS) предназначен для архивации, обработки и быстрого доступа к диагностическим снимкам. Он интегрирован с радиологическим оборудованием, что позволяет врачам оперативно просматривать и анализировать изображения, ставить диагнозы и консультироваться с коллегами без необходимости физического переноса данных.

Сервер радиологической службы поддерживает работу диагностического оборудования, обеспечивает передачу данных между аппаратами и рабочими станциями врачей, а также ведет базу исследований. Это позволяет радиологам оперативно обрабатывать запросы, вести историю болезней пациентов и формировать заключения в едином формате.

Сервер видеонаблюдения обеспечивает безопасность учреждения, записывая и храня данные с камер, установленных в ключевых зонах. Он позволяет контролировать обстановку в реальном времени, расследовать инциденты и предотвращать нарушения, что особенно важно

для соблюдения режима в закрытых помещениях, таких как лаборатории или хранилища лекарств.

Сервер телефонии и почты объединяет корпоративную связь, включая IP-телефонию и электронную почту. Он обеспечивает стабильную коммуникацию между сотрудниками, автоматическую маршрутизацию звонков, запись разговоров и защиту переписки от спама и кибератак.

Сервер расчета зарплаты и отдела кадров автоматизирует процессы начисления заработной платы, учета рабочего времени, налоговых отчислений и кадрового документооборота. Он интегрирован с бухгалтерскими системами, что минимизирует ошибки при расчетах, ускоряет формирование отчетности и упрощает кадровые процедуры, такие как оформление отпусков или больничных.

Организация сети только локальная и соединяется по типу: звезда и шина. Локальная сеть – это замкнутая цифровая среда, в которой устройства объединены для обмена данными и совместного использования ресурсов. Она существует в ограниченном пространстве – например, в пределах здания, офиса или дома – и функционирует как автономная экосистема.

Файловый сервер документооборота обеспечивает централизованное хранение, управление и обмен электронными документами в учреждении. На нём размещаются внутренние приказы, медицинские карты, отчёты, договоры и другие важные файлы в структурированном виде. Сервер поддерживает систему версионности, разграничение прав доступа для разных и резервное копирование данных. Это позволяет сотрудникам быстро находить нужные документы, совместно работать над ними и исключает потерю информации. Интеграция с электронной подписью обеспечивает юридическую значимость документов.

В организации для учёта и управления медицинскими данными используется специализированное программное обеспечение, которое включает следующие программы:

- Электронная медицинская карта «МедЭлемент»;
- Лабораторная информационная система «МедЛаб»;
- Система записи на прием «ЕМИАС»;
- Учет в стационаре «МедОС»;
- Интеграция с диагностическим оборудованием «МедПАКС»;
- Финансовый учет «1С:Бухгалтерия».

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Государственное бюджетное учреждение здравоохранения Республики Хакасия «Абаканская межрайонная клиническая больница» представляет собой крупное многопрофильное медицинское учреждение, которое играет важную роль в системе здравоохранения региона. Больница оказывает специализированную и высокотехнологичную помощь населению, выполняя функции межрайонного центра. Это означает, что учреждение обслуживает не только жителей города Абакана, но и пациентов из близлежащих районов, которые направляются сюда для получения квалифицированного лечения. В структуре больницы работает большое количество различных отделений, каждое из которых оснащено необходимым оборудованием для диагностики, лечения и обеспечения жизнедеятельности пациентов. Для нормальной работы требуется бесперебойное функционирование самой разной техники, начиная от офисных компьютеров и принтеров и заканчивая сложными медицинскими аппаратами и системами связи.

Обеспечение работоспособности технической инфраструктуры возложено на отдел информационных технологий и защиты информации. В штате отдела находятся специалисты разного профиля, включая связистов, инженеров-программистов, специалистов по защите информации и администраторов сети. В их задачи входит поддержка работы семисот клиентских машин и тринадцати серверов, администрирование локальной сети, обеспечение безопасности данных и обслуживание программного обеспечения, такого как медицинские информационные системы и бухгалтерские программы. Сфера ответственности технических специалистов не ограничивается только компьютерной техникой и программным обеспечением. В условиях современного медицинского учреждения практически любое оборудование тем или иным образом связано с общей инфраструктурой, и при возникновении неисправностей сотрудники обращаются именно в этот отдел за помощью или координацией ремонтных работ. Техники выступают в роли единого окна приема заявок, где они фиксируют проблему, определяют исполнителя и контролируют процесс устранения неисправности, независимо от того, требует ли проблема замены компьютерной мыши или ремонта сложного диагностического прибора.

В настоящее время процесс взаимодействия между сотрудниками больницы и техническими специалистами не имеет четкой организационной структуры и не автоматизирован. Когда у работника любого отделения возникает проблема с техникой, он вынужден искать контакты ответственного специалиста самостоятельно. Чаще всего обращение происходит по телефону, через личные сообщения в мессенджерах или устно при личной встрече. Такой способ подачи информации о неисправностях является крайне

неорганизованным и зависит от человеческого фактора. Технический специалист, принимая звонок во время выполнения другой задачи, может не записать детали обращения или забыть перезвонить заявителю. При устных просьбах информация часто передается неточно, что приводит к тому, что мастер приезжает на место без нужных инструментов или запасных частей. В условиях высокой загруженности отдела, когда одновременно поступает множество сигналов о проблемах, некоторые заявки просто теряются в потоке информации и остаются без внимания на неопределенный срок.

Отсутствие единой системы учета заявок создает серьезные трудности не только для сотрудников, ожидающих ремонта, но и для самих технических специалистов. Без централизованного реестра невозможно отследить статус выполнения работ, понять, какая заявка находится в очереди, а какая уже решается. Это приводит к хаосу в планировании рабочего времени инженеров и связистов. Кроме того, при таком подходе теряется история обслуживания оборудования. Если какой-то аппарат ломается регулярно, об этом нет никакой статистики, так как все обращения разрознены и нигде не фиксируются документально. Это мешает руководству отдела принимать обоснованные решения о замене устаревшей техники или проведении профилактических работ. Также возникает проблема с учетом гарантийных обязательств. Без записи даты обращения и истории ремонтов сложно доказать производителю наличие гарантийного случая, что может привести к лишним финансовым расходам больницы на ремонт оборудования, которое должно обслуживаться бесплатно.

Еще одной важной проблемой является отсутствие прозрачности в распределении задач между специалистами. В отделе работают люди разной квалификации, и некоторые проблемы требуют участия узкопрофильных экспертов, не работающих в данной организации.

При текущей схеме подачи заявок через телефон или лично сложно оперативно найти свободного специалиста нужного профиля и передать ему задачу. Информация может передаваться из уст в уста, искажаясь по пути, что увеличивает время простоя оборудования. В медицинской сфере время простоя техники может критически влиять на качество оказания помощи пациентам, поэтому задержки в ремонте недопустимы. Кроме того, отсутствует контроль за использованием запасных частей и материалов. Когда ремонт выполняется без фиксации в системе, сложно понять, какие детали были потрачены, и своевременно пополнить складские запасы, что в будущем может привести к невозможности проведения ремонта из-за отсутствия комплектующих.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО МОДУЛЯ

3.1 Разработка диаграмм IDEF0 и DFD

В результате процесса «учет и обработка заявок на ремонт техники» необходимы входные данные, такие как: «данные о неисправности оборудования», «информация об оборудовании», «данные сотрудника», «описание неисправности».

Для того, чтобы получить на выходе «зарегистрированная заявка с уникальным номером», «уведомление о статусе заявки», «отчет о выполненных работах», «история ремонтов оборудования».

Под управлением регламентирующих документов – «регламент работы IT отдела», «политики информационной безопасности», «должностные инструкции специалистов», «ГОСТ Р 7.0.97-2016», «положение об отделе информационных технологий».

Механизмами, которые взаимодействуют с процессами, являются: «сотрудники организации (заказчики услуг)», «программное обеспечение (веб-система учета, БД)», «IT инфраструктура (700 ПК, 13 серверов, сеть)», «технические специалисты (связисты, инженеры-программисты)», «склад запасных частей и инструментов», «справочники (оборудование, сотрудники, приоритеты)». Посмотреть можно в соответствии с рисунком 3.1.

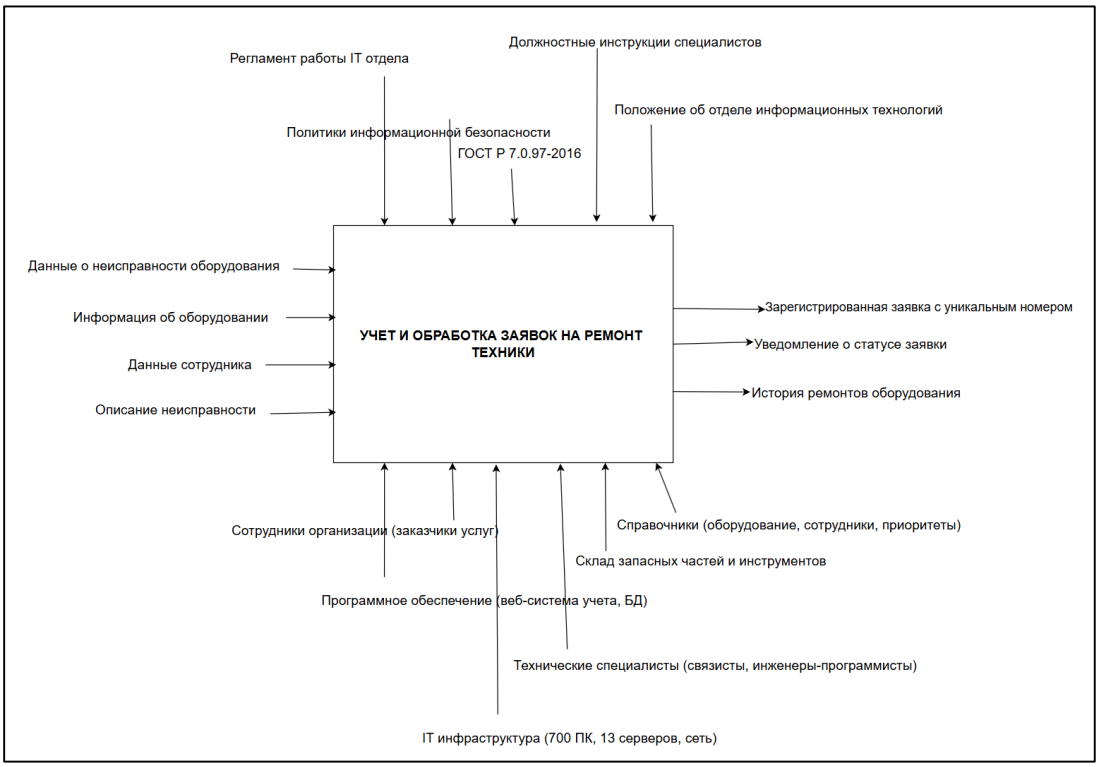


Рисунок 3.1 – IDEF0 нулевого уровня

В результате декомпозиции IDEF0 диаграммы 0 уровня происходит декомпозиция блока «учет и обработка заявок на ремонт техники» на 4 подпроцесса: «заполнение формы заявки», «отправка заявки технику», «обработка заявки» и «подтверждение и сохранение заявки». В результате распределения входных данных по подпроцессам было получено следующее:

Входные данные: «данные о неисправности оборудования», «информация об оборудовании», «данные сотрудника», «описание неисправности» входят в подпроцесс «заполнение формы заявки»;

Входные данные «заполненная форма заявки» входят в подпроцесс «отправка заявки технику»;

Входные данные «заявка с данными сотрудника» входят в подпроцесс «обработка заявки»;

Входные данные «заявка с данными оборудования» входят в подпроцесс «подтверждение и сохранение заявки».

Управляющая документация «регламент работы IT отдела» входит в подпроцессы «заполнение формы заявки» и «обработка заявки»;

Управляющая документация «должностные инструкции специалистов» и «политики информационной безопасности» входят в подпроцесс «заполнение формы заявки»;

Управляющая документация «ГОСТ Р 7.0.97-2016» входит в подпроцесс «обработка заявки»;

Управляющая документация «положение об отделе информационных технологий» входит в подпроцесс «подтверждение и сохранение заявки».

Механизмы «сотрудники организации (заказчики услуг)» и «справочники (оборудование, сотрудники, приоритеты)» принимают участие в подпроцессе «заполнение формы заявки»;

Механизм «технические специалисты (связисты, инженеры-программисты)» входит в подпроцесс «отправка заявки технику»;

Механизмы «программное обеспечение (веб-система учета, БД)», «IT инфраструктура (700 ПК, 13 серверов, сеть)» и «склад запасных частей и инструментов» входят в подпроцесс «обработка заявки»;

Механизмы «программное обеспечение (веб-система учета, БД)» и «справочники (оборудование, сотрудники, приоритеты)» входят в подпроцесс «подтверждение и сохранение заявки».

Благодаря этим входным данным, управляющей документации и механизмам, мы получим на выходе: «зарегистрированная заявка с уникальным номером», «уведомление о статусе заявки» и «история ремонтов оборудования» в соответствии с рисунком 3.2.

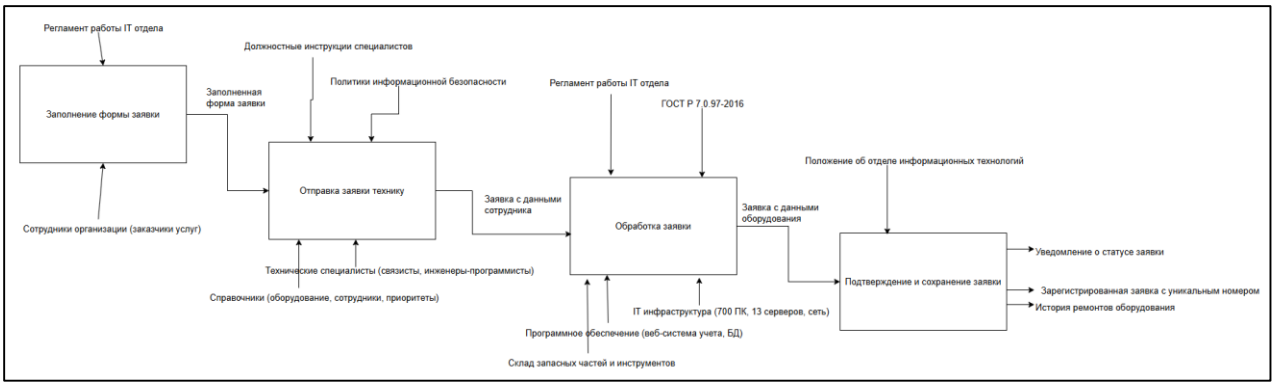


Рисунок 3.2 – IDEF0 первого уровня

Посмотреть созданную ER-диаграмму можно в соответствии с рисунком 3.3.

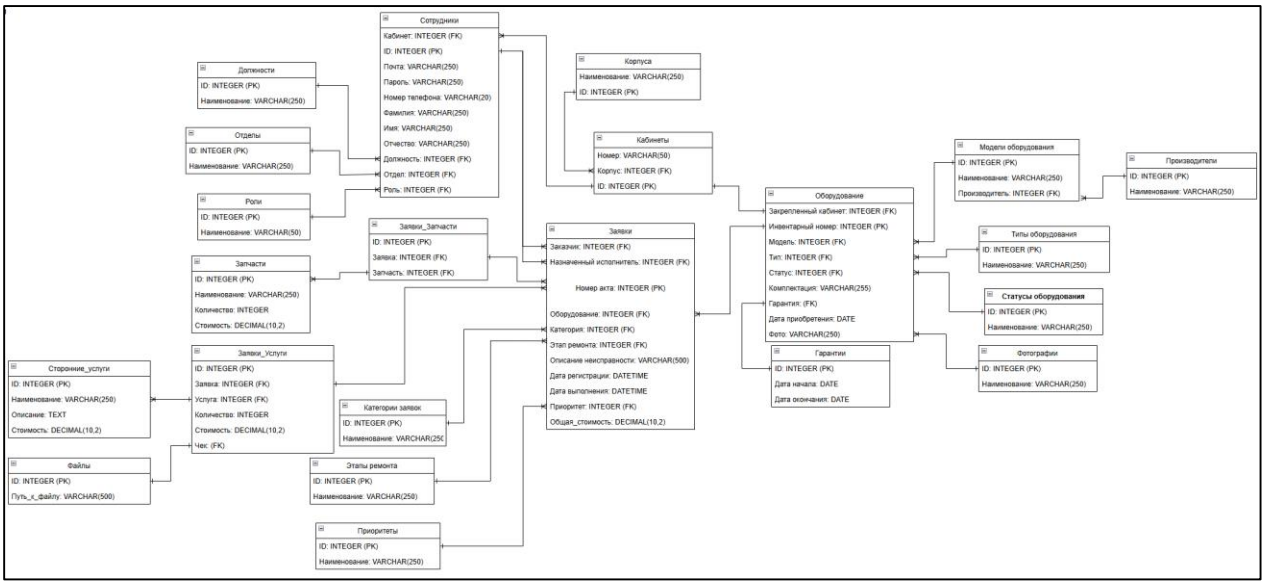


Рисунок 3.3 – ER-диаграмма

ER-диаграмма состоит из 19 таблиц:

Должности включает в себя «ID» и «Наименование». «ID» — первичный ключ. Должность может быть у одного или нескольких сотрудников, при этом обязательно есть.

Отделы включает в себя «ID» и «Наименование». «ID» — первичный ключ. Отдел может включать как одного, так и нескольких сотрудников, при этом обязательно привязан к сотрудникам.

Роли включает в себя «ID» и «Наименование». «ID» — первичный ключ. Роль может быть как у одного, так и у нескольких сотрудников и обязательно есть.

Корпуса включает в себя «ID» и «Наименование». «ID» — первичный ключ. Корпус может включать как один, так и несколько кабинетов, при этом обязательно присутствует.

Кабинеты состоит из «ID» — первичный ключ, «Номер» и «Корпус» — внешний ключ. Кабинету соответствует только один корпус и обязательно соответствует. В кабинете может быть закреплено одно оборудование, несколько или вообще не быть оборудования. В кабинете может быть несколько сотрудников.

Сотрудники состоит из следующих полей: «ID» — первичный ключ, «Номер телефона» «Почта», «Пароль», «Фамилия», «Имя», «Отчество», «Должность» — внешний ключ, «Отдел» — внешний ключ, «Кабинет» — внешний ключ, «Роль» — внешний ключ. Сотруднику соответствует только одна должность, один отдел, один кабинет и одна роль, при этом обязательно соответствует. Сотрудник может быть как заказчиком, так и назначенным исполнителем в одной или нескольких заявках.

Производители включает в себя «ID» и «Наименование». «ID» — первичный ключ. Производитель может быть как у одной, так и у нескольких моделей оборудования и обязательно есть.

Модели оборудования состоит из «ID» — первичный ключ, «Наименование» и «Производитель» — внешний ключ. Модели соответствует только один производитель и обязательно соответствует. Модель может быть как у одного, так и у нескольких оборудований.

Типы оборудования включает в себя «ID» и «Наименование». «ID» — первичный ключ. Тип оборудования может быть как у одного, так и у нескольких оборудований и обязательно есть.

Статусы оборудования включает в себя «ID» и «Наименование». «ID» — первичный ключ. Статус может быть как у одного, так и у нескольких оборудований и обязательно есть.

Фотографии включает в себя «ID» и «Наименование». «ID» — первичный ключ. Фотография может быть как у одного, так и у нескольких оборудований и обязательно есть.

Оборудование состоит из «Закрепленный кабинет» — внешний ключ, «Инвентарный номер» — первичный ключ, «Модель» — внешний ключ, «Тип» — внешний ключ, «Статус» — внешний ключ, «Фото» — внешний ключ, «Комплектация», «Гарантия» — внешний ключ, «Дата приобретения». Оборудованию соответствует только один кабинет, одна модель, один тип, один статус, одно фото и обязательно соответствует. Оборудование может быть в одной или нескольких заявках.

Гарантии состоит из «ID» — первичный ключ, «Дата начала», «Дата окончания». Гарантия есть у каждого оборудования и у каждого она своя.

Категории заявок включает в себя «ID» и «Наименование». «ID» — первичный ключ. Категория может быть как у одной, так и у нескольких заявок и обязательно есть.

Этапы ремонта включает в себя «ID» и «Наименование». «ID» — первичный ключ. Этап ремонта может быть как у одной, так и у нескольких заявок и обязательно есть.

Приоритеты включает в себя «ID» и «Наименование». «ID» — первичный ключ. Приоритет может быть как у одной, так и у нескольких заявок и обязательно есть.

Запчасти состоит из «ID» — первичный ключ, «Наименование», «Количество», «Стоимость». Запчасть может быть как в одной, так и в нескольких заявках.

Заявки_Запчасти состоит из «ID» — первичный ключ, «Заявка» — внешний ключ, «Запчасть» — внешний ключ. Связывает заявки и запчасти в соотношении многие-ко-многим.

Заявки состоит из «Заказчик» — внешний ключ, «Назначенный исполнитель» — внешний ключ, «Номер акта» — первичный ключ, «Оборудование» — внешний ключ, «Категория» — внешний ключ, «Этап ремонта» — внешний ключ, «Описание неисправности», «Дата регистрации», «Дата выполнения», «Приоритет» — внешний ключ. Заявке соответствует только один заказчик, только один исполнитель, только одно оборудование, только одна категория, только один этап ремонта, только один приоритет и обязательно соответствует.

Так же была разработана DFD диаграмма или же диаграмма потоков данных.

На диаграмме потоков данных отображено взаимодействие внешних сущностей сотрудников и технических специалистов с процессами системы учета заявок на ремонт техники. Процесс авторизации принимает от внешних сущностей сотрудников и техники учетные данные в виде логина и пароля для проверки прав доступа путем обращения к хранилищу сотрудники содержащему список сотрудников в системе. После успешной проверки пользователь системы направляется на процесс просмотра заявок где формируется список всех заявок в системе доступный для ознакомления.

Технические специалисты обладают расширенными правами техника, позволяющими инициировать процесс создания заявки путем передачи данных заявки на вход процесса где используется информация из хранилищ справочники и оборудование для корректного заполнения полей формы. Результатом работы процесса создания заявки является внесение заявки в базу к остальным записям хранилища заявки, а также передача данных заявки на процесс формирования отчетов по заявкам для последующей аналитики.

Кроме того, технические специалисты взаимодействуют с процессом редактирования заявки передавая данные заявки для изменения существующих записей с использованием данных оборудования и используемой информации из справочников что обеспечивает актуальность сведений в системе. Все информационные потоки связывают внешние сущности процессы обработки данных и хранилища в единую структуру обеспечивающую учет и обработку обращений по неисправностям техники. Посмотреть диаграмму можно в соответствии с рисунком 3.4.

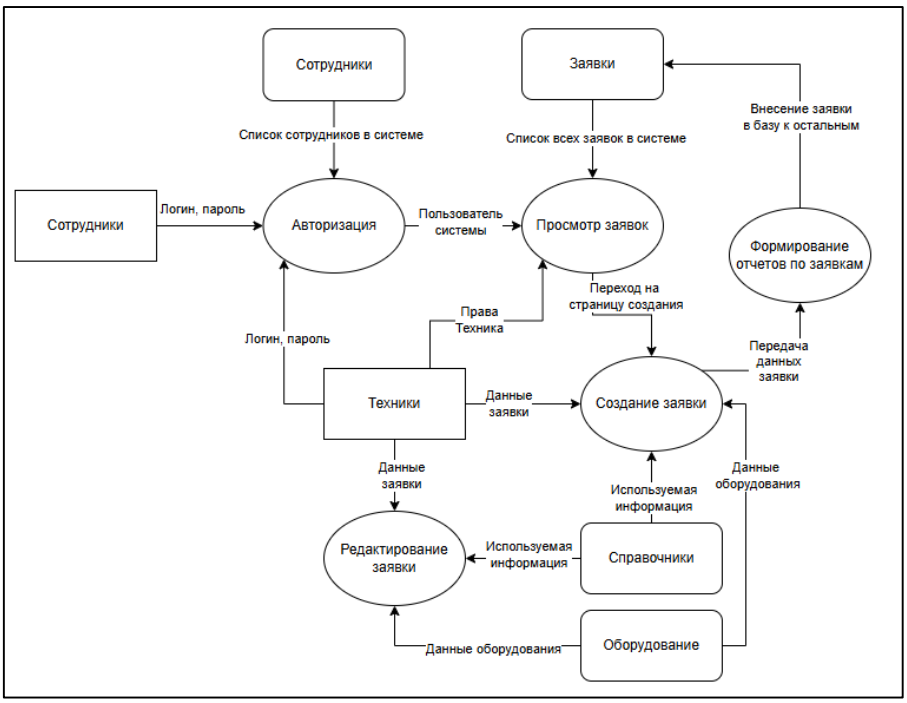


Рисунок 3.4 – DFD диаграмма 0 уровня

3.2 Репозиторий в системе контроля версий

Git — это распределённая система контроля версий, которая необходима для отслеживания изменений в исходном коде проекта, координации работы разработчиков и обеспечения сохранности данных.

Для его создания сначала проводится регистрация на сайте, а после переход на вкладку репозиториев. На вкладке нажимается кнопка «new», заполняются поля, которые необходимы и помечается создание «readme» файла. Посмотреть создание репозитория можно в соответствии с рисунком 3.5.

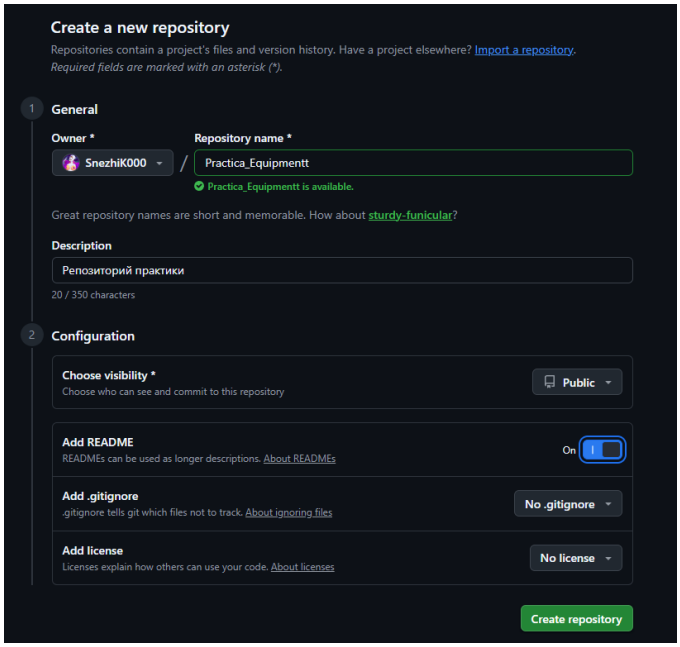


Рисунок 3.5 – Создание репозитория

Посмотреть окно созданного репозитория можно в соответствии с рисунком 3.6

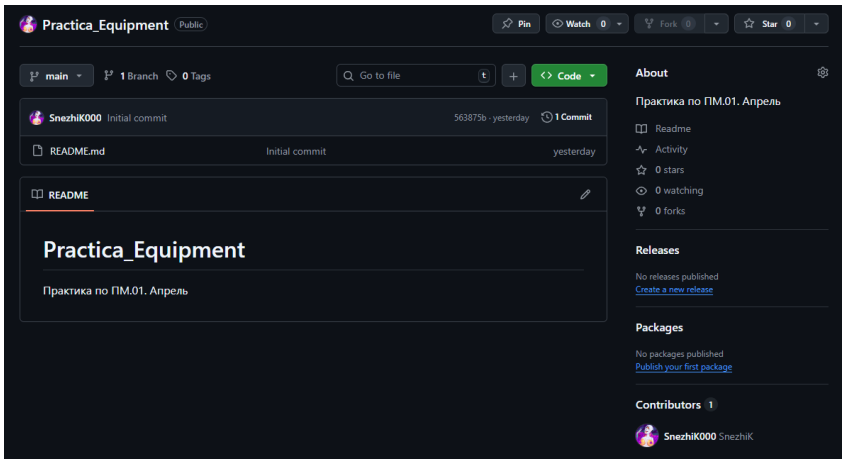


Рисунок 3.6 – Окно репозитория

3.3 Разработка макетов интерфейса программного модуля

Перед разработкой были созданы макеты интерфейса, по которым разрабатывалась программа учета заявок. Первым была разработана страница входа в приложение. Посмотреть страницу входа можно в соответствии с рисунком 3.7.

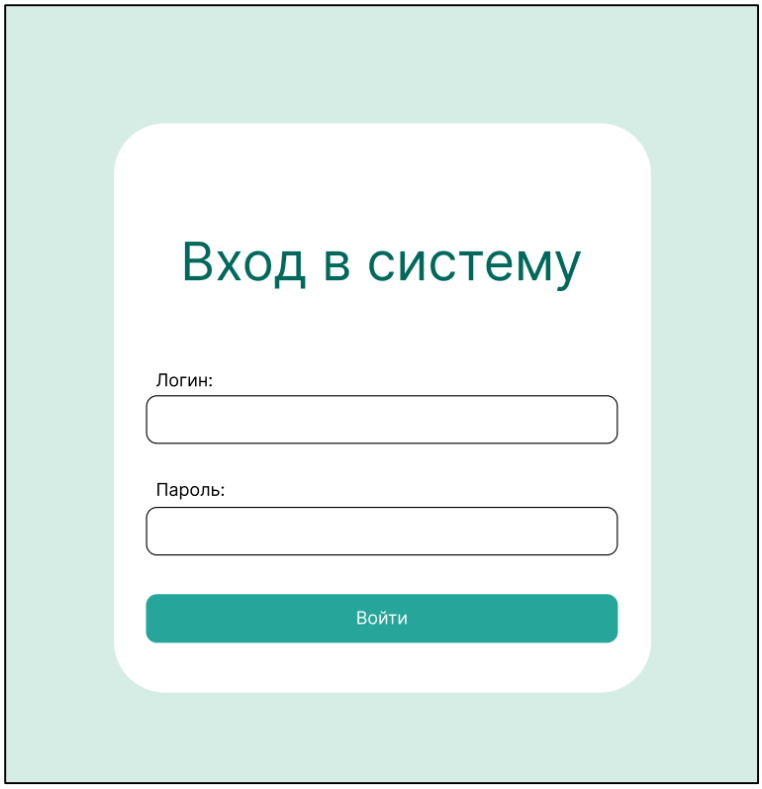


Рисунок 3.7 – Макет интерфейса входа в систему

Макет страницы заявок сотрудника можно посмотреть в соответствии с рисунком 3.8.

Рисунок 3.8 – Макет интерфейса заявок сотрудника

Макет страницы создания заявки сотрудником можно посмотреть в соответствии с рисунком 3.9.

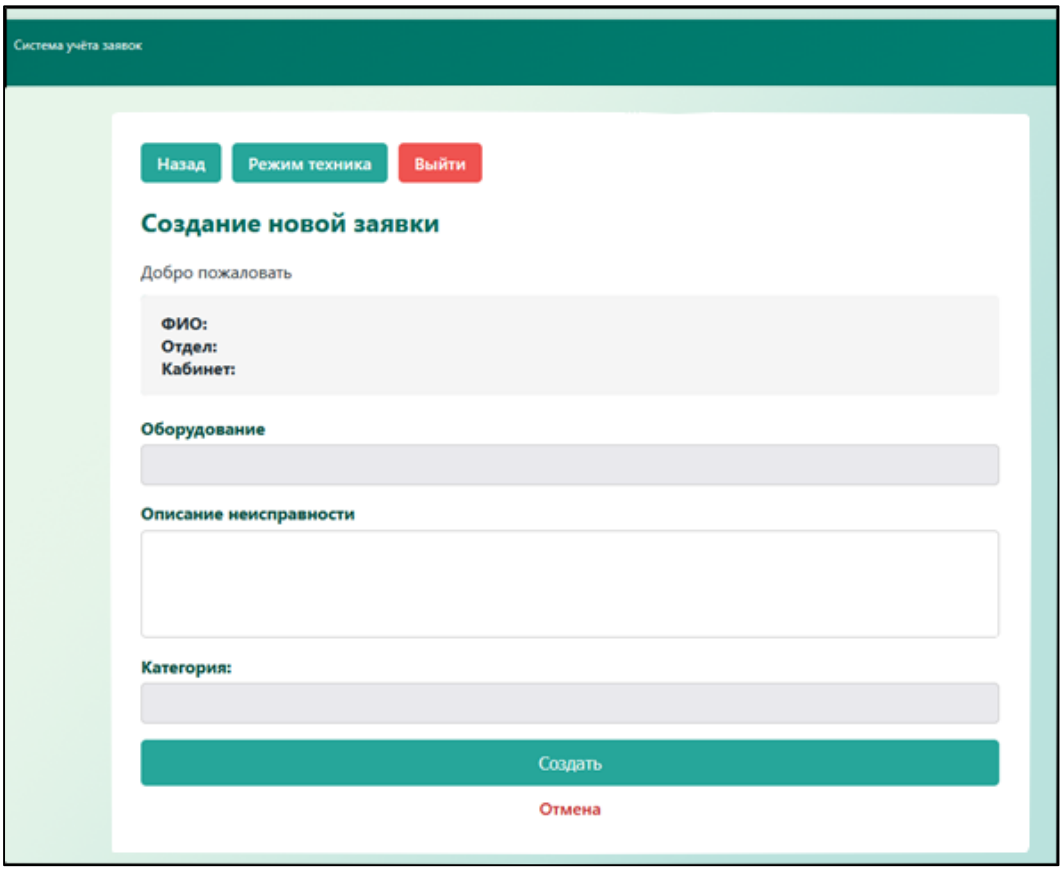


Рисунок 3.9 – Макет интерфейса создания заявки сотрудником

Макет страницы отображения всех заявок под ролью «техник», можно посмотреть в соответствии с рисунком 3.10.

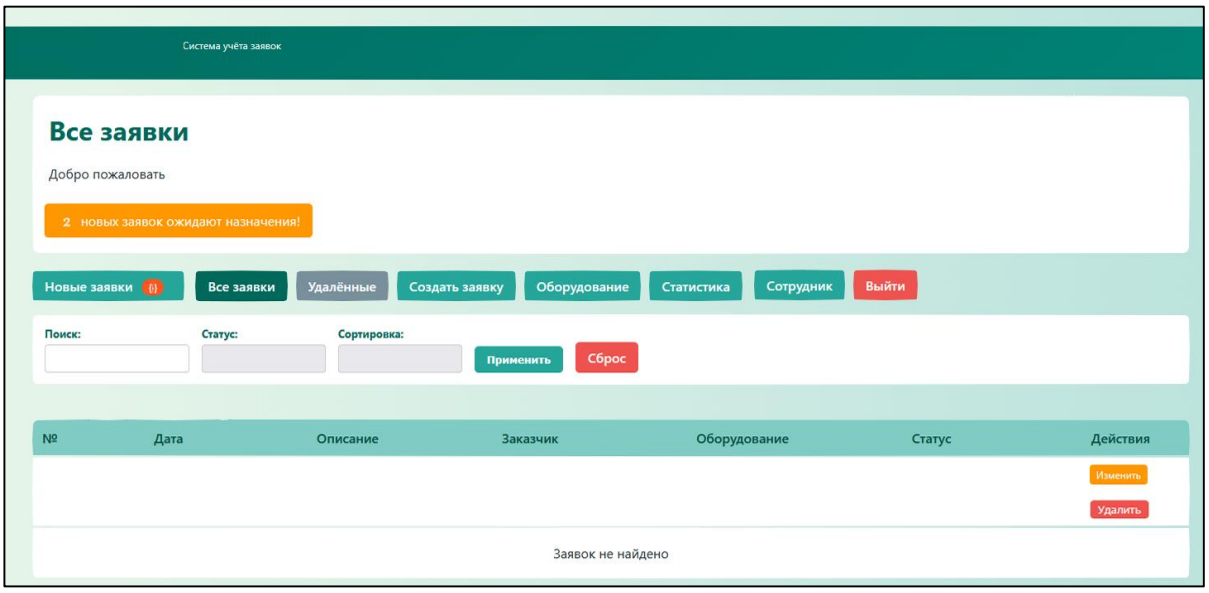


Рисунок 3.10 – Макет интерфейса отображения заявок под ролью «техник»

Макет страницы отображения новых заявок от сотрудников можно посмотреть в соответствии с рисунком 3.11.

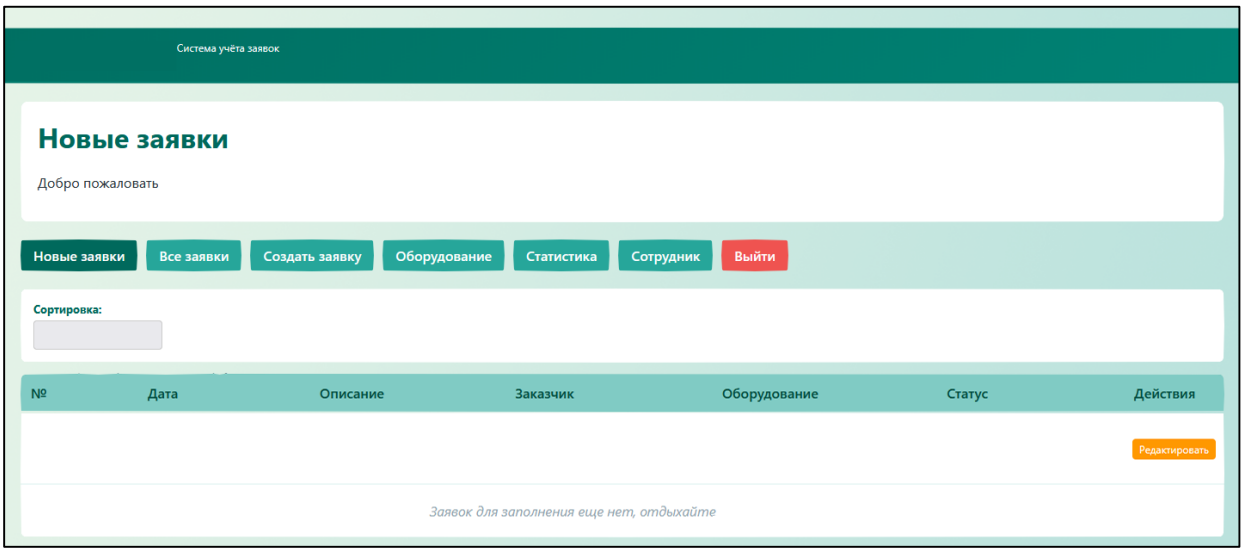


Рисунок 3.11 – Макет интерфейса отображения новых заявок под ролью «техник»

Макет страницы отображения мягко удаленных заявок от сотрудников можно посмотреть в соответствии с рисунком 3.12.

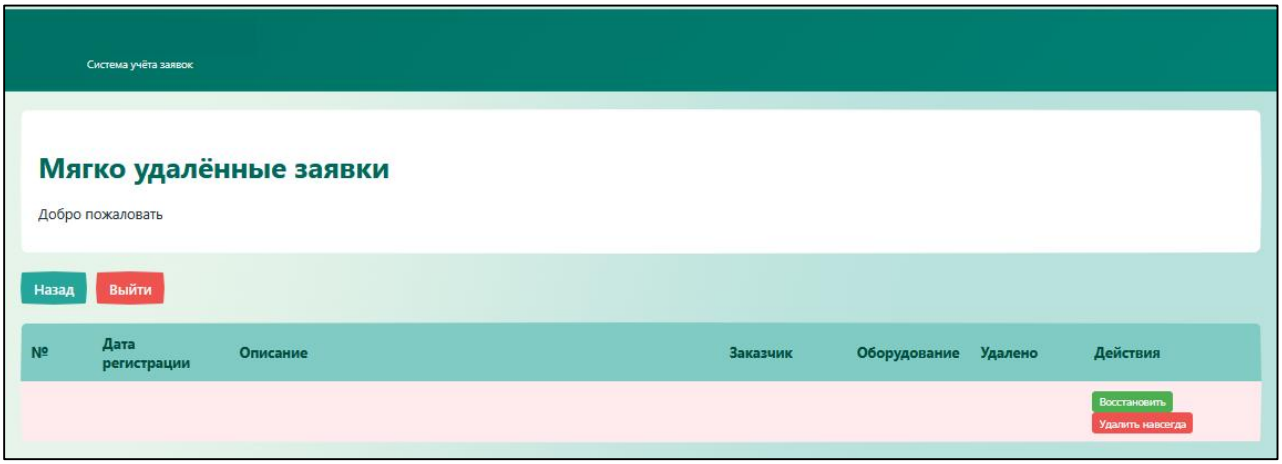


Рисунок 3.12 – Макет интерфейса отображения мягко удаленных заявок

Макет страницы отображения оборудования можно посмотреть в соответствии с рисунком 3.13.

Рисунок 3.13 – Макет интерфейса отображения оборудования

Макет страницы отображения затрат на всё оборудование можно посмотреть в соответствии с рисунком 3.14.

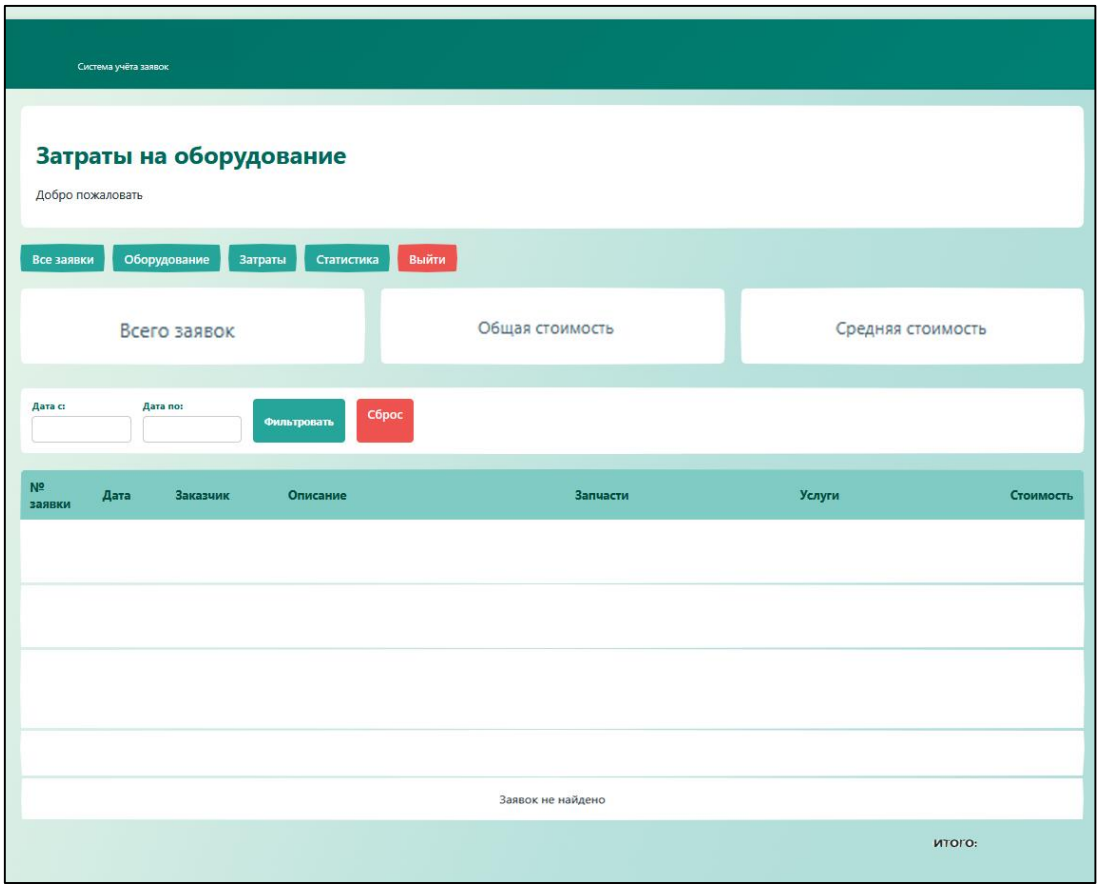


Рисунок 3.14 – Макет интерфейса отображения затрат на все оборудование

Макет страницы редактирования заявки изображен в соответствии с рисунком 3.15.

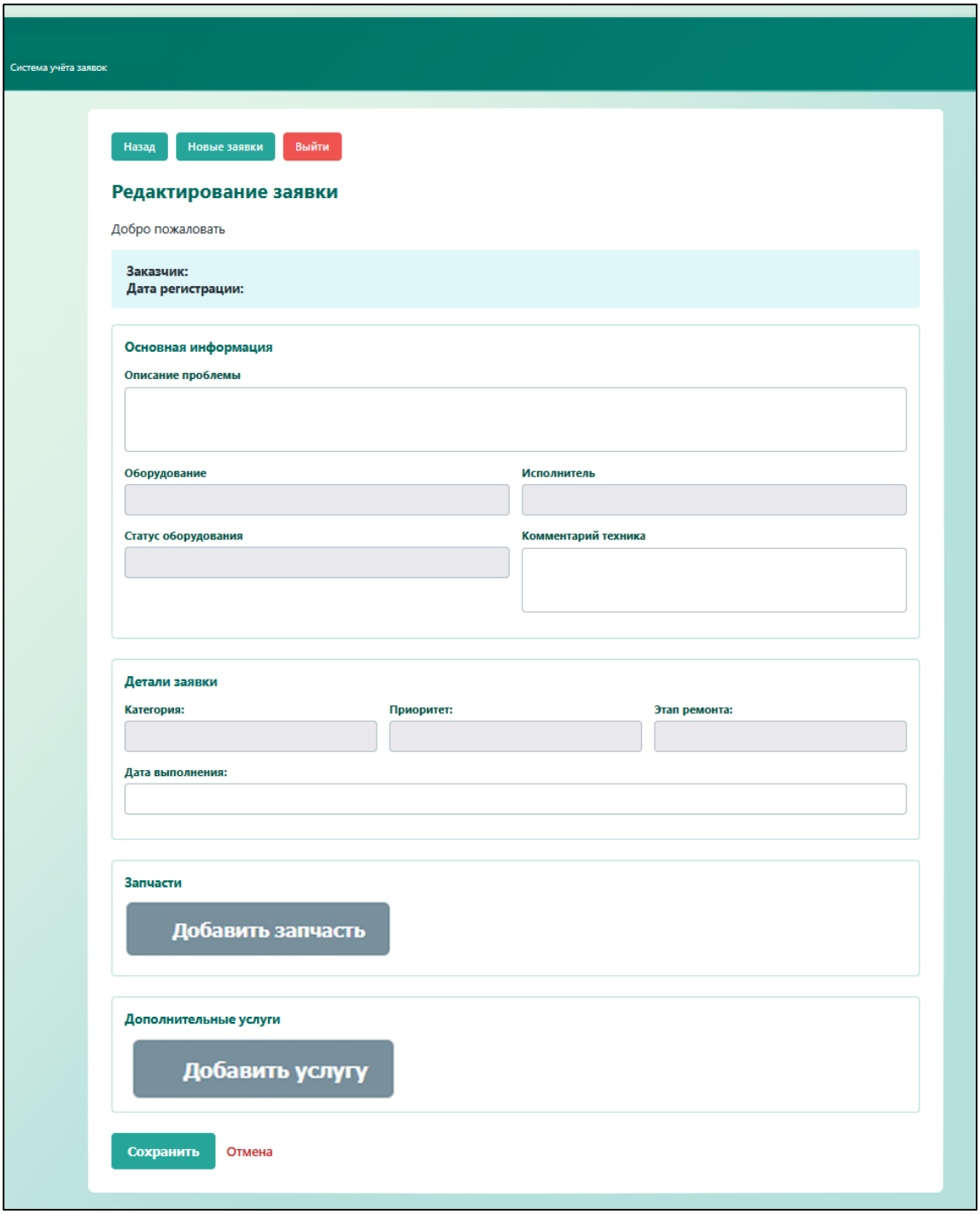


Рисунок 3.15 – Макет интерфейса редактирования заявки

Макет страницы создания заявки техником можно посмотреть в соответствии с рисунком 3.16.

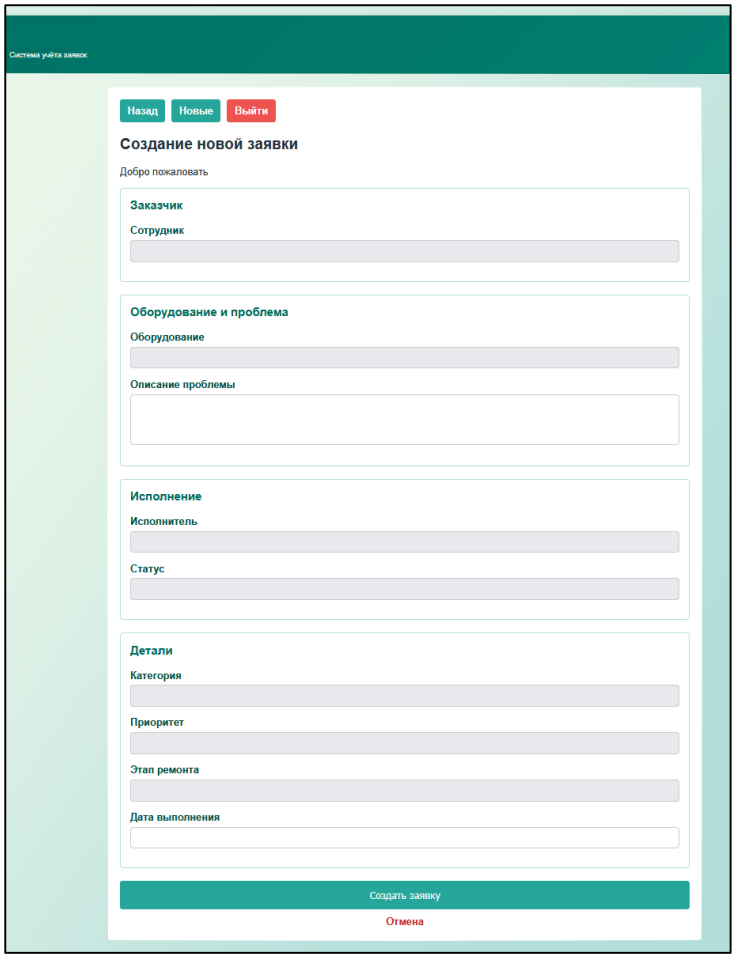


Рисунок 3.16 – Макет интерфейса создания заявки техником

Макет страницы просмотра статистики по ремонтам можно посмотреть в соответствии с рисунком 3.17.

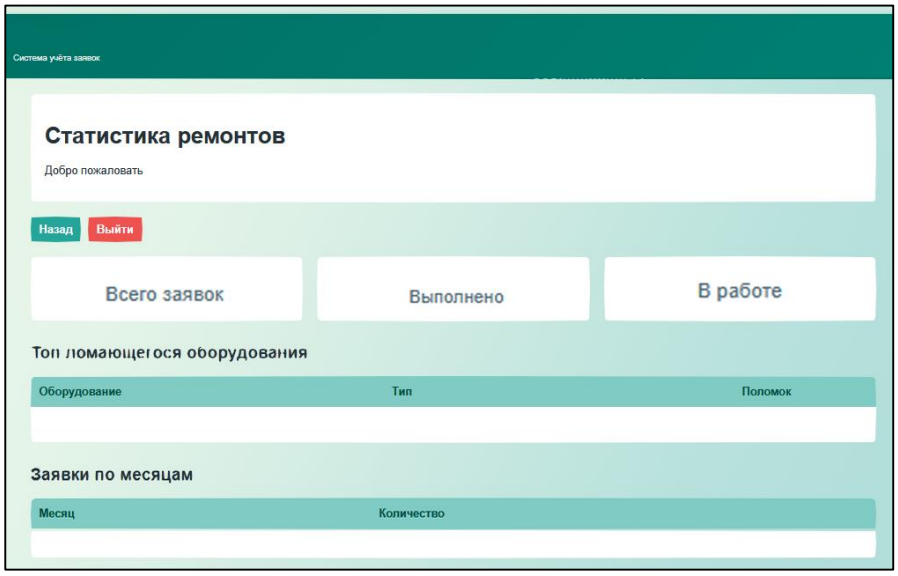


Рисунок 3.17 – Макет интерфейса статистики по ремонтам

4 РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

4.1. Разработка интерфейсов программного модуля

Страница авторизации пользователя предназначена для входа в систему сотрудников организации с различными ролями доступа. На странице расположены поля для ввода логина и пароля, кнопка подтверждения входа, а также блок для отображения сообщений об ошибках аутентификации.

После успешного ввода учётных данных происходит перенаправление пользователя на соответствующую страницу в зависимости от назначенной роли — техника или заказчика. Посмотреть страницу авторизации можно в соответствии с рисунком 4.1.

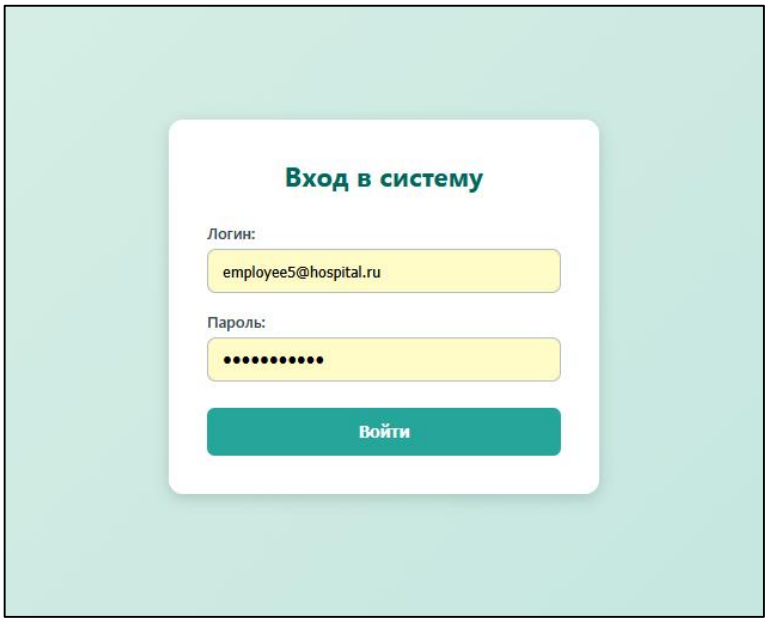


Рисунок 4.1 – Страница входа в систему

Страница личного кабинета сотрудника отображает список всех заявок, созданных авторизованным пользователем в роли заказчика. На странице реализован функционал поиска по описанию проблемы, номеру заявки или инвентарному номеру оборудования, фильтрации по статусам оборудования, а также сортировки по дате регистрации или номеру акта. Для каждой заявки отображается основная информация: номер акта, дата регистрации, описание неисправности, оборудование, статус и назначенный исполнитель. Удаление заявки доступно только для выполненных обращений. Посмотреть страницу личного кабинета сотрудника можно в соответствии с рисунком 4.2.

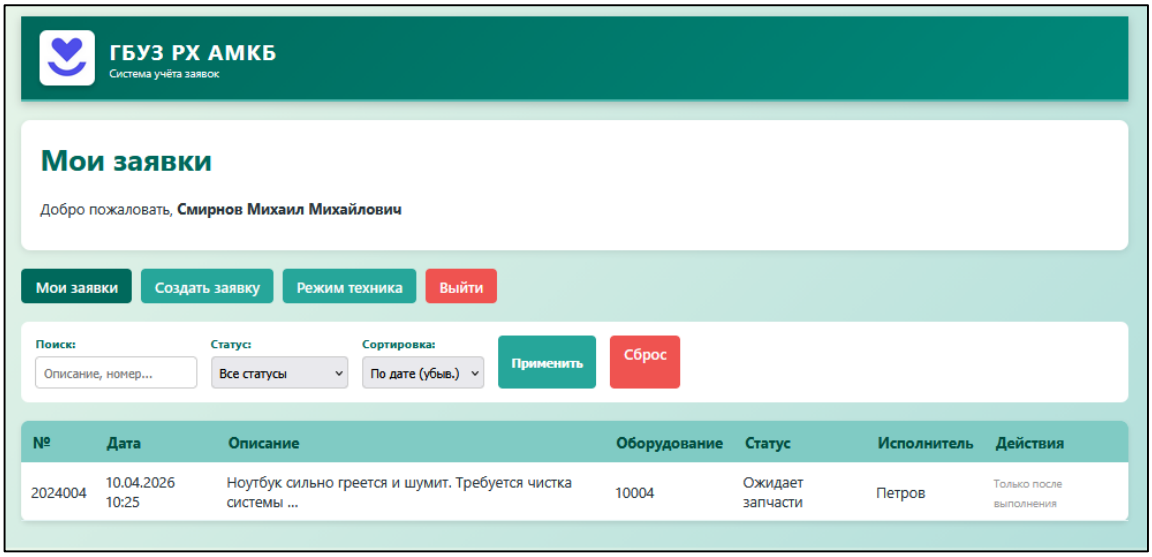


Рисунок 4.2 – Страница сотрудника

Страница создания новой заявки предоставляет сотруднику возможность оформить обращение на ремонт техники. На странице отображается информация о текущем пользователе, а также форма с полями для выбора оборудования из списка, закреплённого за кабинетом сотрудника, описания неисправности и категории проблемы. Обязательными для заполнения являются поля оборудования и описания проблемы. После успешного создания заявки происходит автоматическое присвоение номера акта и даты регистрации, затем пользователь перенаправляется в личный кабинет. Посмотреть страницу создания заявки можно в соответствии с рисунком 4.3.

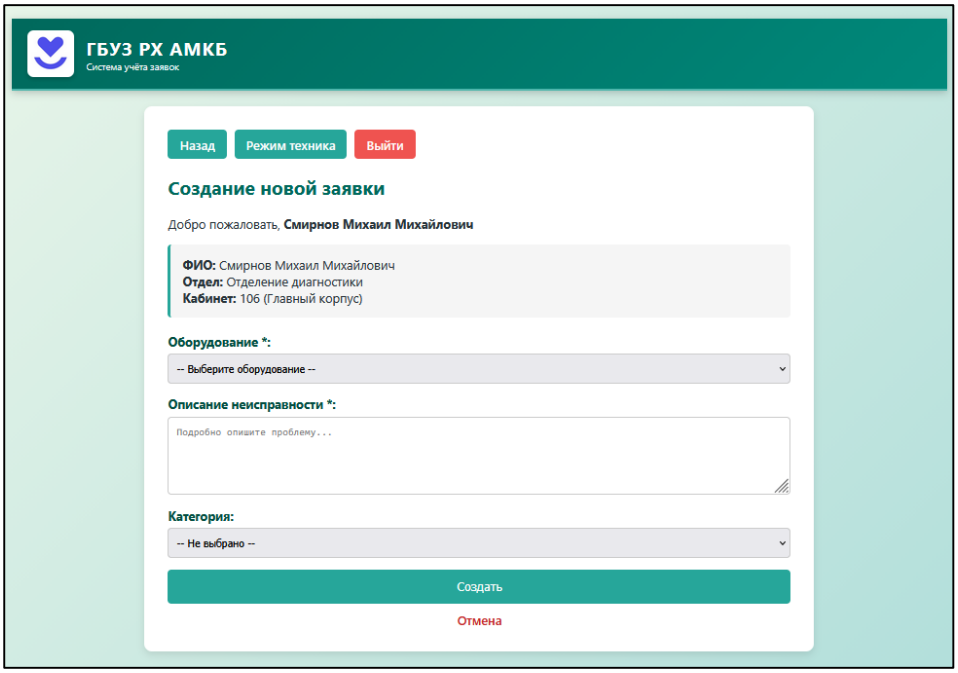


Рисунок 4.3 – Страница создания заявки от роли сотрудник

Страница «Все заявки» предназначена для отображения полного списка заявок системы для авторизованного пользователя с ролью «Техник». На странице реализован функционал поиска по описанию проблемы, номеру акта, фамилии заказчика или инвентарному номеру оборудования, фильтрации по статусам оборудования, а также сортировки по дате регистрации, номеру акта или имени заказчика. Для каждой заявки отображается основная информация: номер акта, дата регистрации, описание неисправности, заказчик, оборудование, статус и кнопки действий. Уведомление о количестве новых заявок отображается в верхней части страницы. Посмотреть страницу можно в соответствии с рисунком 4.4.

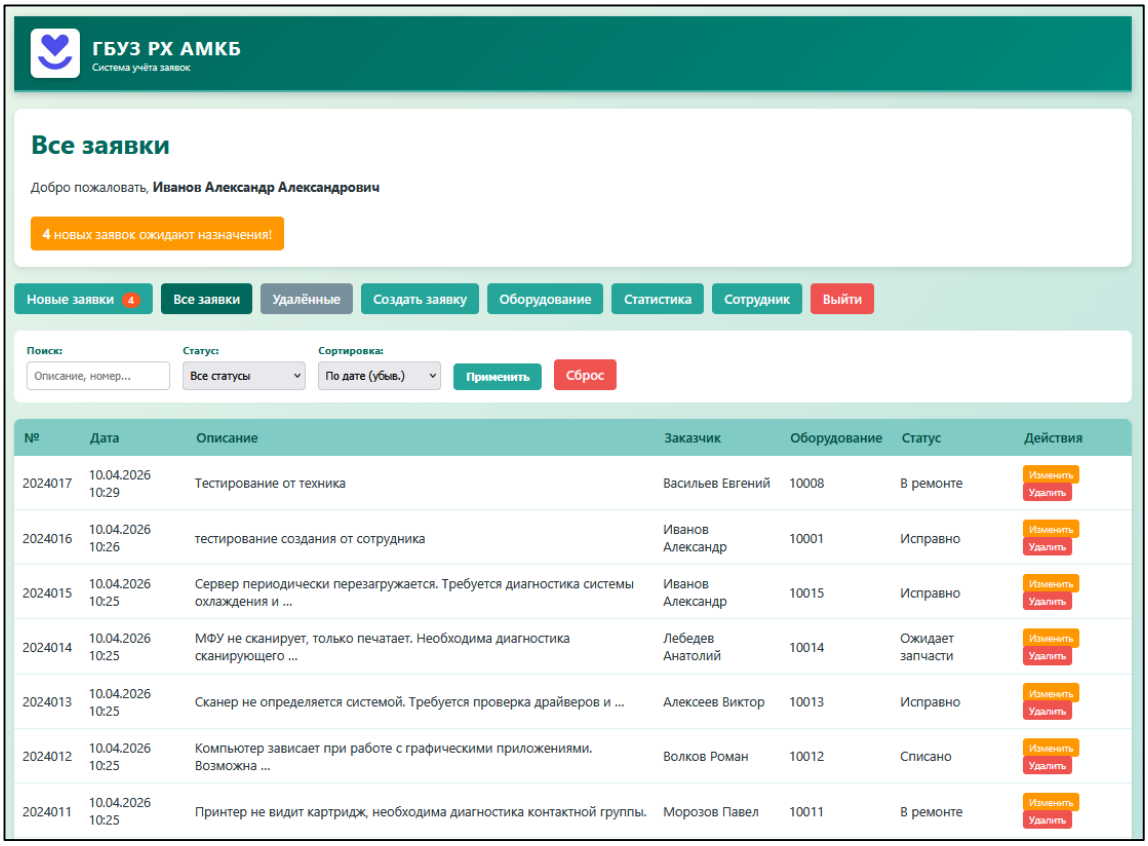


Рисунок 4.4 – Страница техника

Страница «Новые заявки» предназначена для отображения только тех заявок, которые ещё не назначены на исполнителя или не имеют даты выполнения. Данная страница позволяет техническому специалисту быстро определить заявки, требующие первоочередного внимания и назначения исполнителя. На странице реализована сортировка по дате регистрации, номеру акта или заказчику. Для каждой заявки доступна кнопка редактирования для назначения исполнителя и изменения статуса. Посмотреть страницу можно в соответствии с рисунком 4.5.

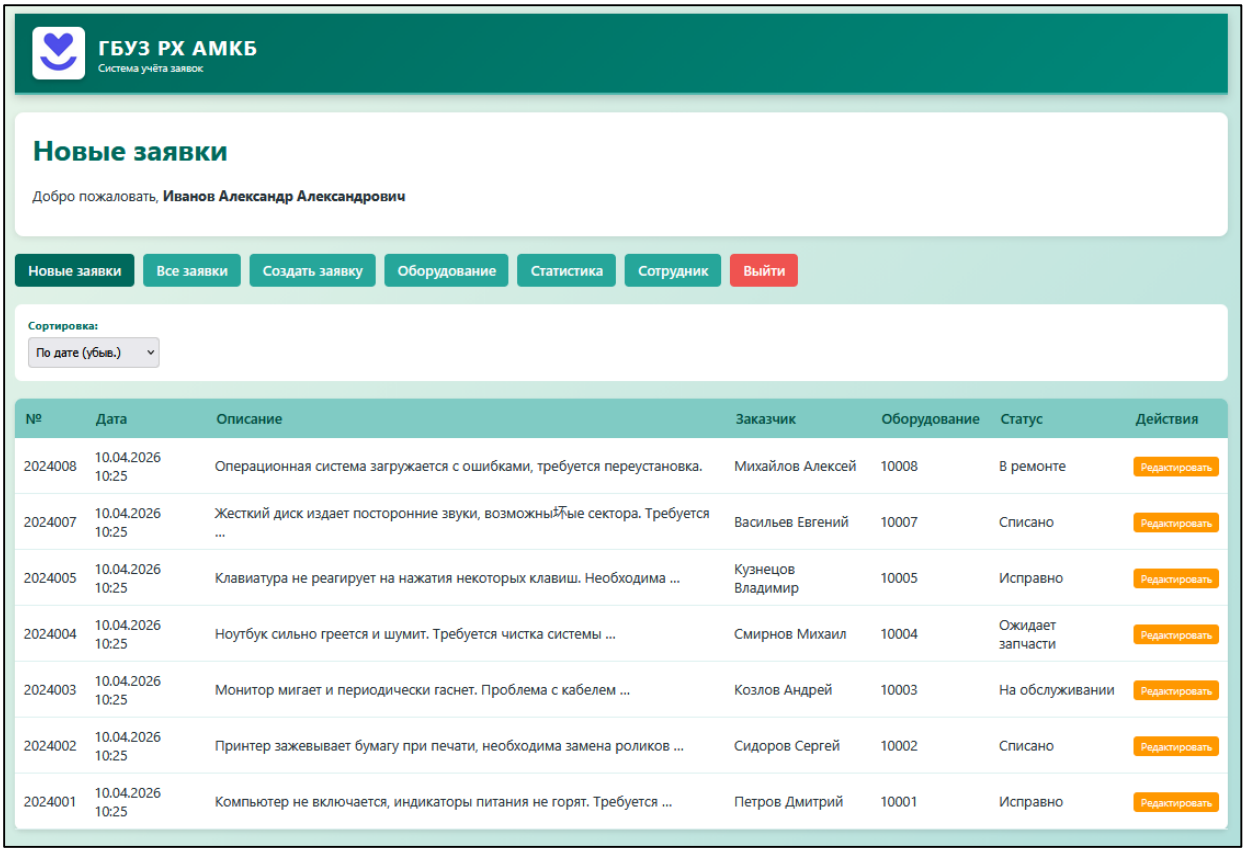


Рисунок 4.5 – Страница новых заявок

Страница «Мягко удалённые заявки» предназначена для отображения заявок, которые были мягко удалены из системы. На странице отображается дата удаления для каждой заявки. Технический специалист имеет возможность восстановить заявку из архива путём нажатия кнопки «Восстановить», что возвращает заявку в активный список, или удалить заявку навсегда путём нажатия кнопки «Удалить навсегда». Посмотреть страницу можно в соответствии с рисунком 4.6.

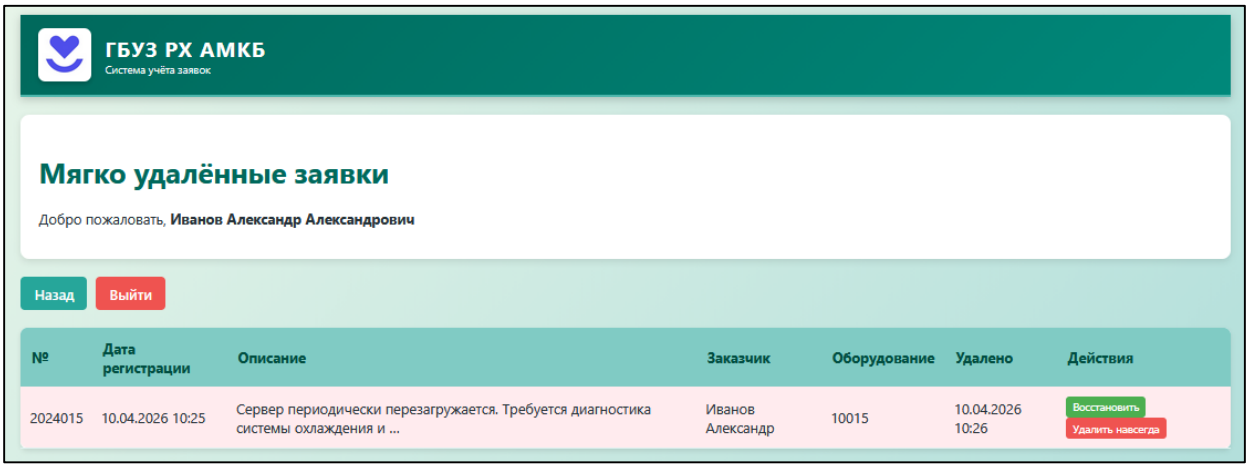


Рисунок 4.6 – Страница мягко удаленных заявок

Страница «Оборудование» предназначена для отображения списка всего оборудования организации с подробной информацией по каждой единице техники. На странице реализован функционал поиска по инвентарному номеру или модели оборудования, фильтрации по типу и статусу, а также сортировки по различным параметрам. Для каждой позиции оборудования отображается фотография, инвентарный номер, модель, тип, комплектация, закрепленный кабинет, статус и информация о гарантии. Доступны кнопки для редактирования информации об оборудовании и просмотра затрат на обслуживание. Посмотреть страницу можно в соответствии с рисунком 4.7.

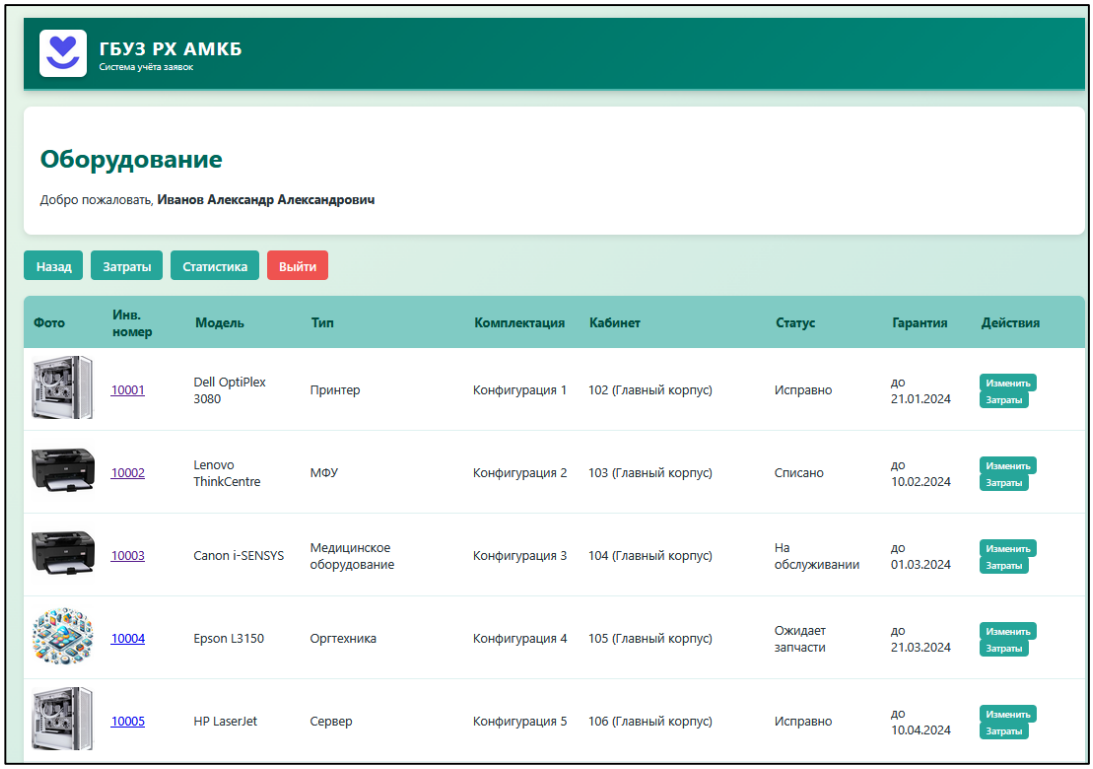


Рисунок 4.7 – Страница мягко удаленных заявок

Страница «Затраты на оборудование» предназначена для расчёта и отображения общей стоимости обслуживания конкретного оборудования или всего парка техники за выбранный период. На странице отображается сводная информация: количество заявок, общая стоимость всех работ, средняя стоимость одной заявки.

Для каждой заявки детализируются использованные запчасти и услуги с указанием количества и стоимости. Реализован функционал фильтрации по периоду дат регистрации заявок. Посмотреть страницу затрат для всего оборудования можно в соответствии с рисунком 4.8.

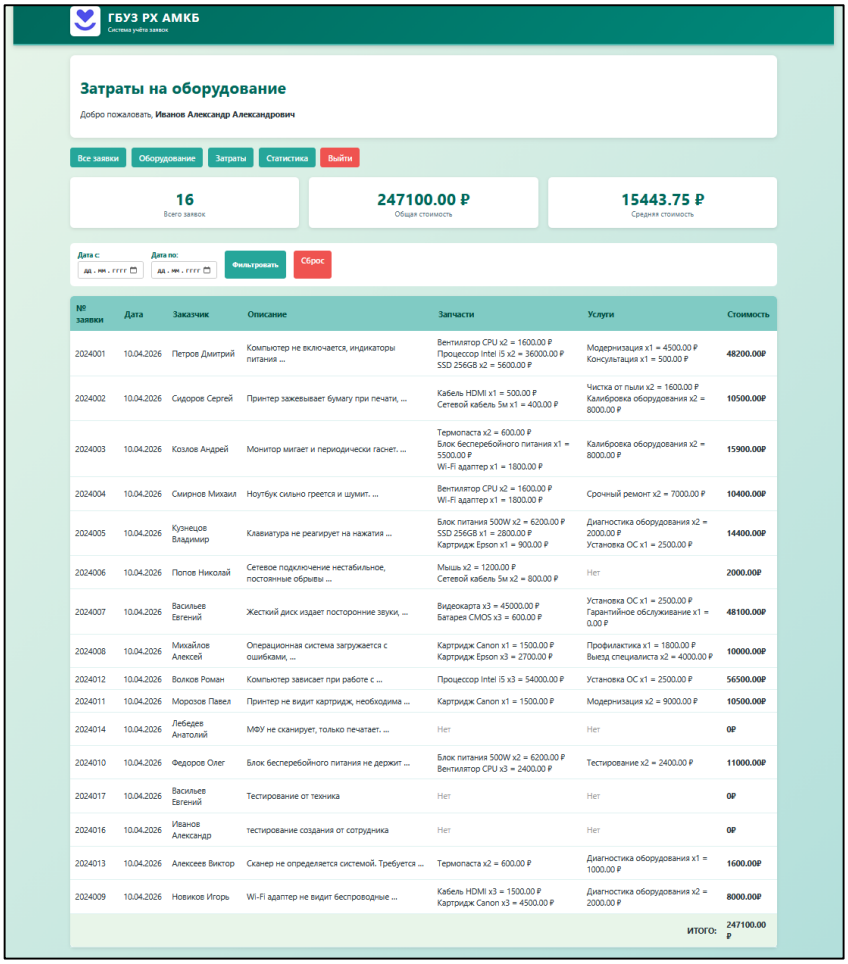


Рисунок 4.8 – Страница затрат на все оборудование в общем

Посмотреть отображение для конкретного оборудования можно в соответствии с рисунком 4.9.

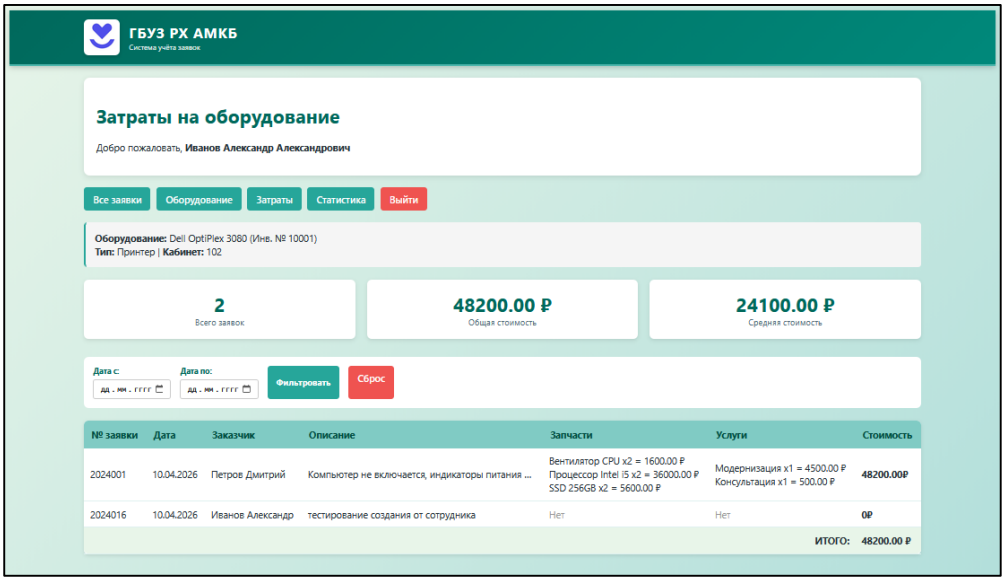


Рисунок 4.9 – Страница затрат на одно конкретное оборудование за весь период

Страница «Редактирование заявки» предназначена для изменения существующей заявки техническим специалистом. На странице отображается основная информация о заявке: номер акта, заказчик, дата регистрации. Технический специалист имеет возможность изменить описание проблемы, выбрать оборудование, назначить исполнителя, установить статус оборудования, добавить комментарий, выбрать категорию, приоритет и этап ремонта, указать дату выполнения. Реализован функционал добавления использованных запчастей с указанием количества и стоимости, а также дополнительных услуг с прикреплением чеков об оплате. Посмотреть страницу можно в соответствии с рисунком 4.10.

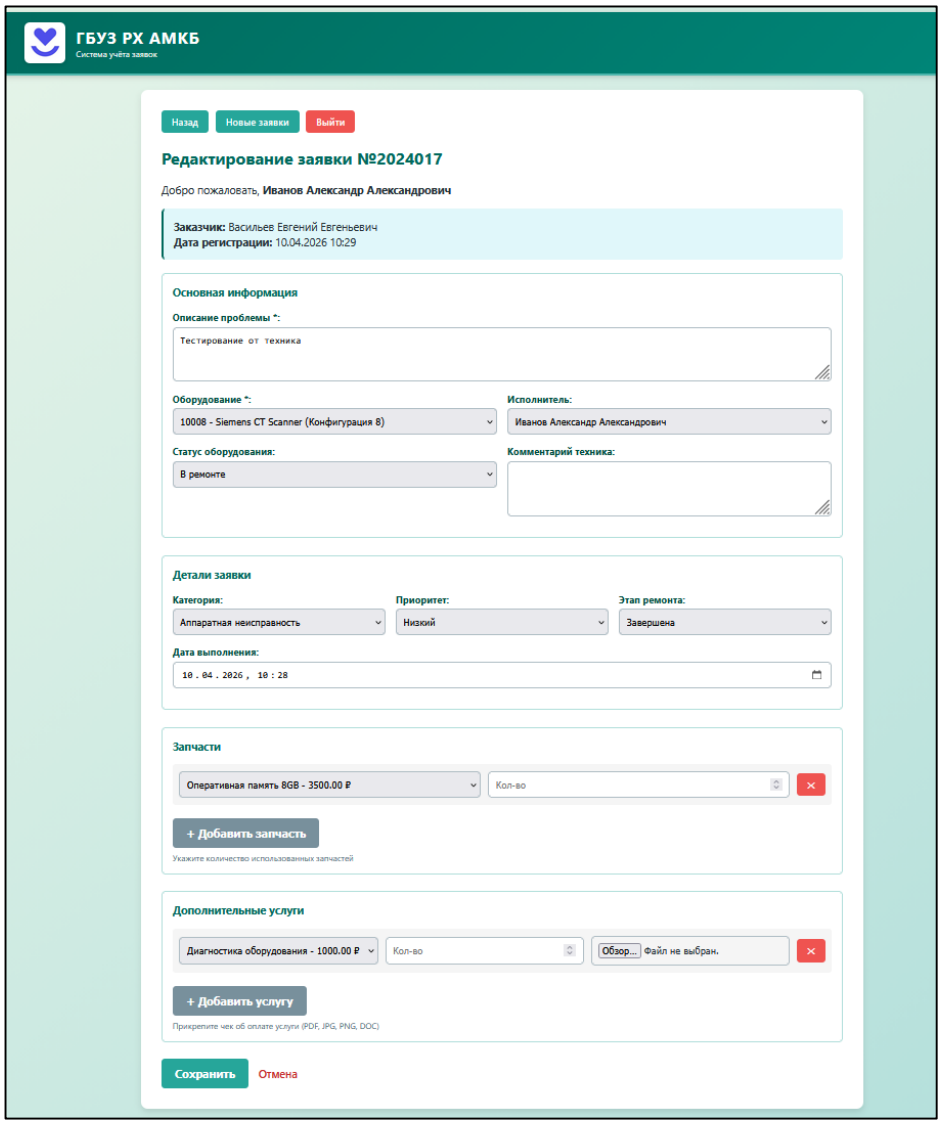


Рисунок 4.10 – Страница редактирования заявки техником

Страница «Создание заявки техником» предоставляет техническому специалисту возможность создания новой заявки от имени любого сотрудника организации. На странице

реализованы поля для выбора заказчика из списка всех сотрудников, выбора оборудования из общего парка техники, назначения исполнителя, указания описания проблемы. Дополнительно доступны поля для выбора статуса оборудования, категории заявки, приоритета, этапа ремонта и даты выполнения. При сохранении заявки автоматически генерируется номер акта и фиксируется дата регистрации. Посмотреть страницу можно в соответствии с рисунком 4.11.

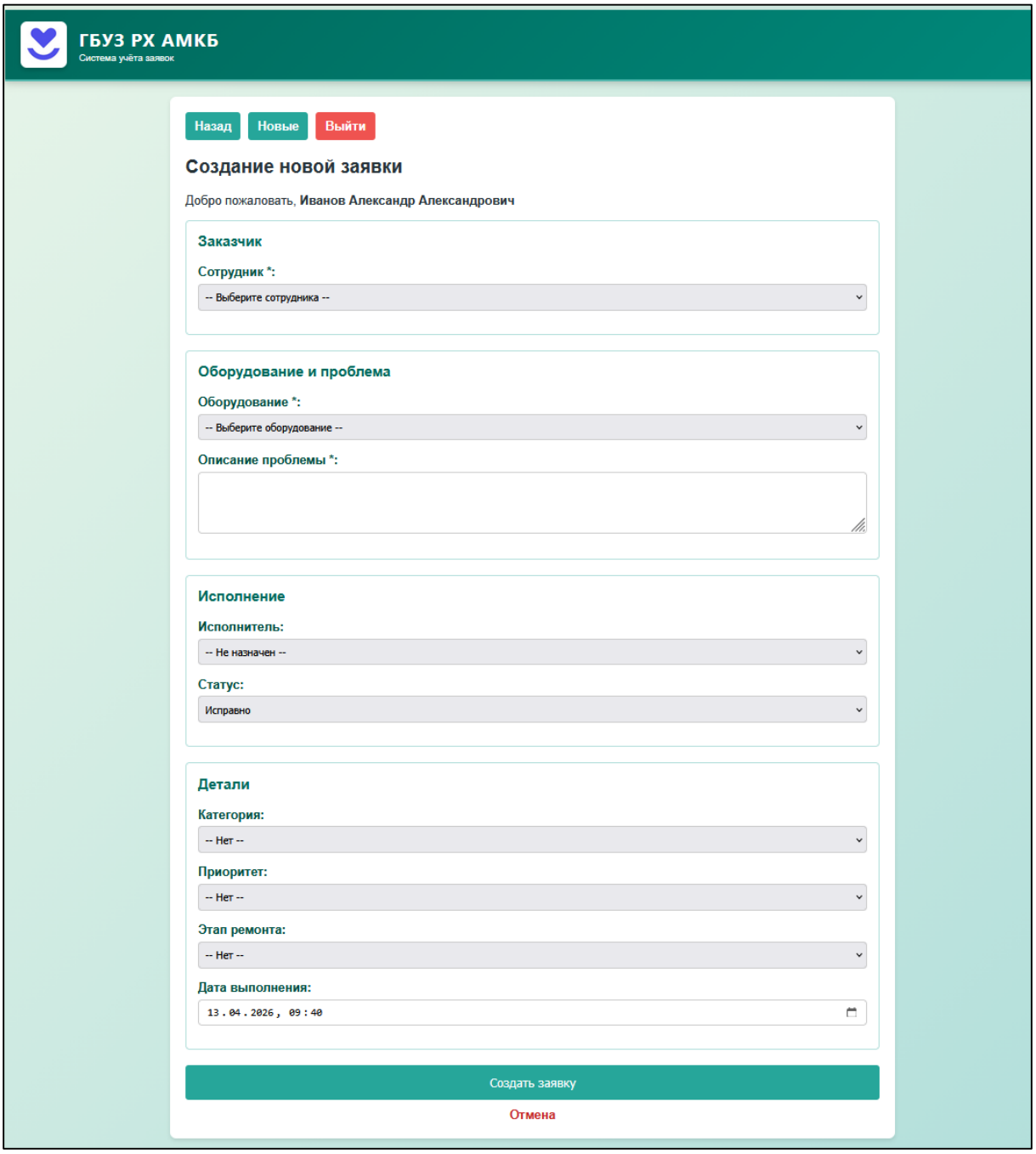


Рисунок 4.11 – Страница создания заявки техником

Страница «Редактирование оборудования» предназначена для изменения информации об единице техники техническим специалистом. На странице отображается инвентарный номер оборудования и текущие параметры: модель, тип, статус, закрепленный кабинет, комплектация. Реализован функционал загрузки новой фотографии оборудования с автоматическим сохранением в директорию media и присвоением уникального имени файла. При загрузке нового фото предыдущее помечается как удалённое с сохранением в архиве. Посмотреть страницу можно в соответствии с рисунком 4.12.

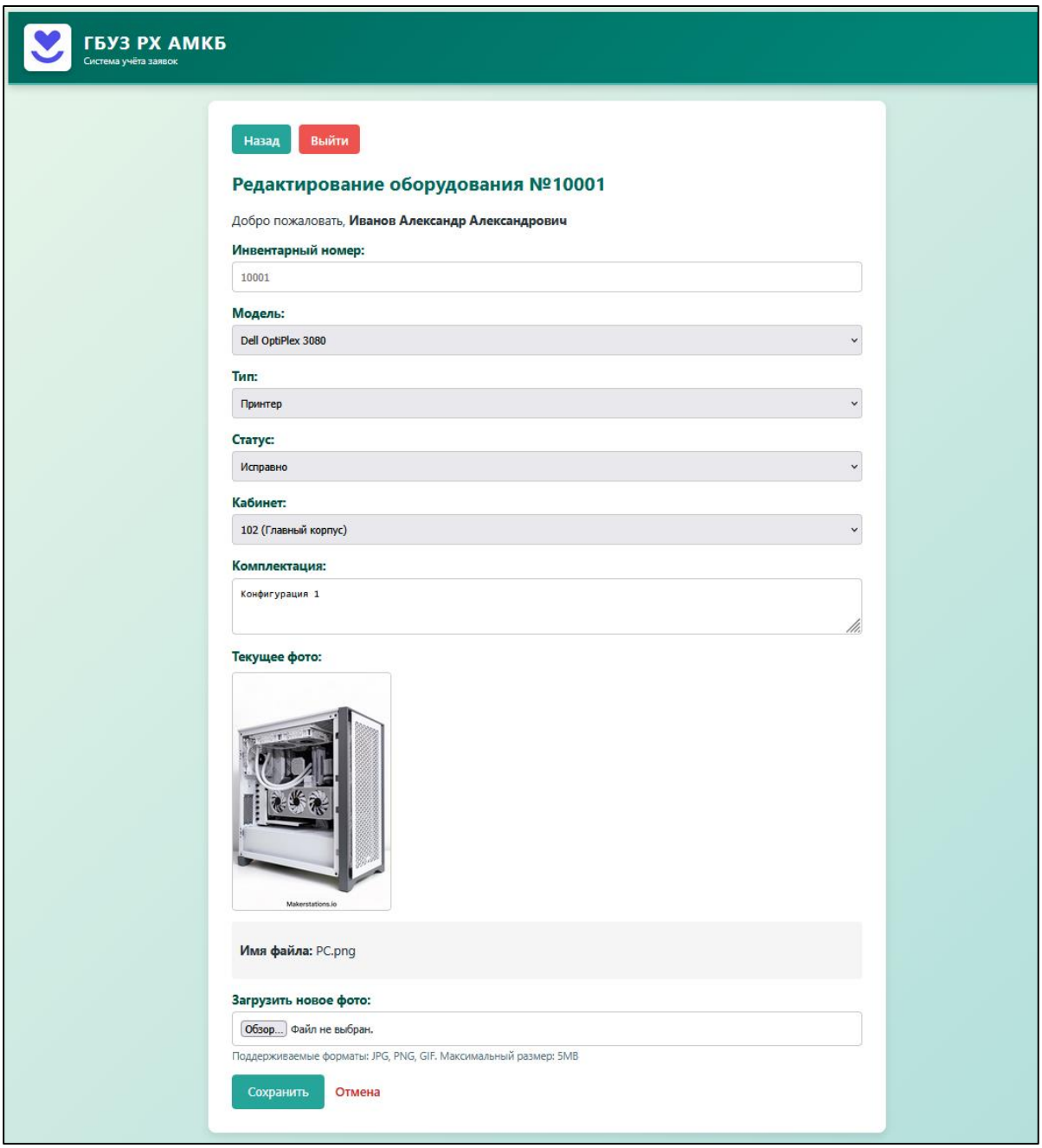


Рисунок 4.12 – Страница редактирования оборудования техником

Страница «Статистика» предназначена для отображения аналитической информации по заявкам на ремонт техники. На странице представлены сводные данные: общее количество заявок, количество выполненных заявок, количество заявок в работе. Реализован вывод топ-10 оборудования по количеству поломок с указанием типа оборудования и количества обращений. Доступна статистика по месяцам за последние полгода с количеством зарегистрированных заявок. Посмотреть страницу можно в соответствии с рисунком 4.13.

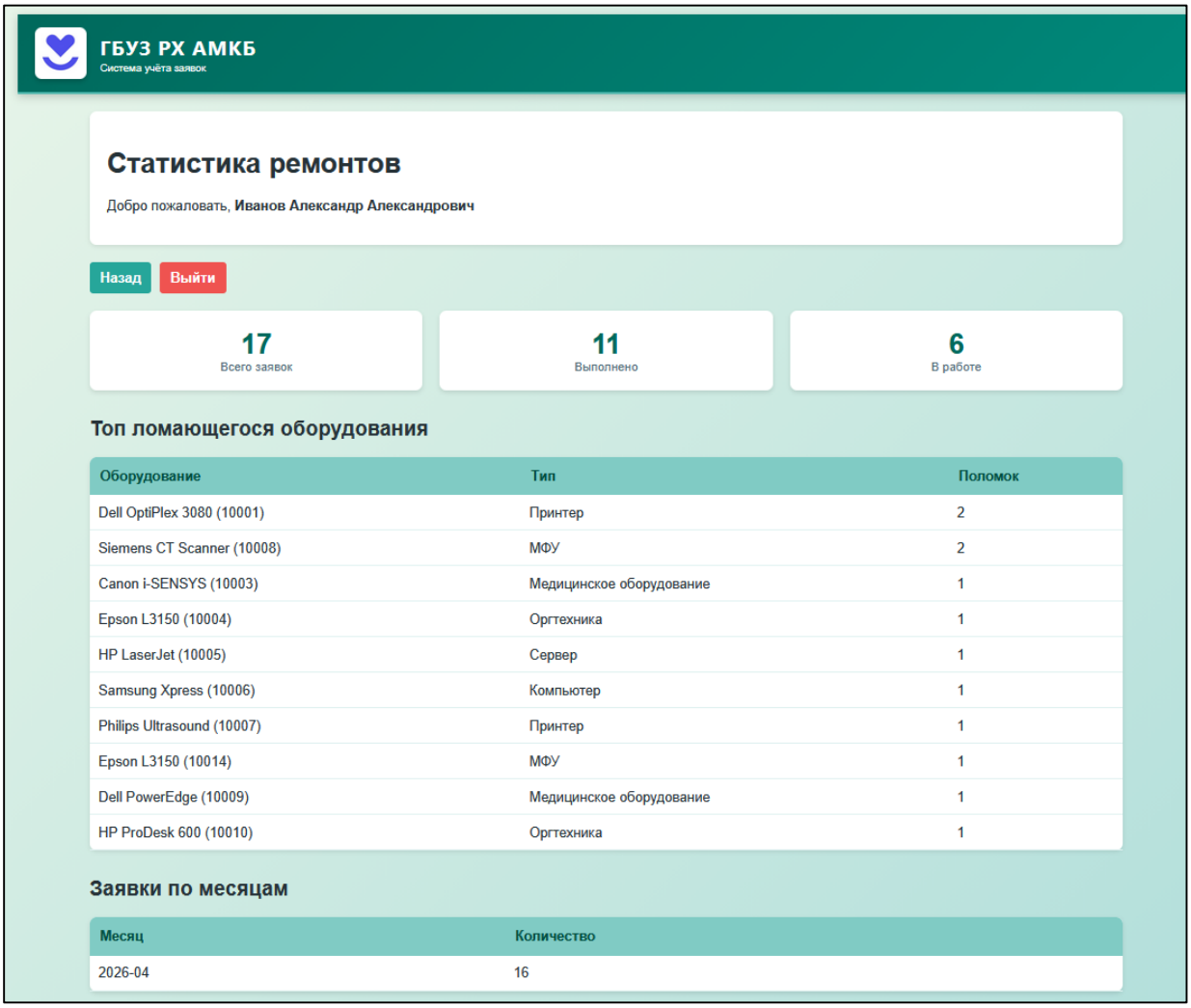


Рисунок 4.13 – Страница редактирования оборудования техником

4.2. Разработка кода программного модуля

Функция login_view обеспечивает аутентификацию пользователя в системе путем проверки введенного логина и пароля, при этом реализация выполнена через прямое сравнение пароля в открытом виде без хеширования, успешный вход фиксируется в сессии с сохранением роли и имени пользователя для последующего перенаправления на страницу

техника или сотрудника в зависимости от должности. Код реализации представлен на листинге 1.

Листинг 1 – Login_view

```
def login_view(request):
    if request.method == 'POST':
        login_input = request.POST.get("username")
        password = request.POST.get("password")
        try:
            user      =      Employee.objects.get(login=login_input,
delete_date__isnull=True)
            if user.password == password:
                request.session['user_id'] = user.id
                request.session['user_role'] = user.role.role_name
                request.session['user_name']  =  f'{user.last_name}
{user.first_name}'
                user.last_login = timezone.now()
                user.save()
                if user.role.role_name == 'Техник':
                    return redirect('show_technician')
                return redirect('show_customer')
        except Employee.DoesNotExist:
            messages.error(request, 'Пользователь не найден')
    return render(request, 'login.html')
```

Функция logout_view обеспечивает выход пользователя из системы путем очистки сессии и удаления всех сохраненных данных авторизации, после чего осуществляется перенаправление на страницу входа. Код реализации представлен на листинге 2.

Листинг 2 – Logout_view

```
def logout_view(request):
    request.session.flush()
    messages.info(request, "Вы вышли из системы!")
    return redirect('login_view')
```

Функция CustomerRequestListView отображает личный кабинет сотрудника, формируя выборку заявок только для авторизованного пользователя, и реализует функционал поиска по тексту проблемы или оборудованию через Q-объекты, а также фильтрацию по статусам заявки. Код реализации представлен на листинге 3.

Листинг 3 – CustomerRequestListView

```
class CustomerRequestListView(ListView):
    model = RequestFix
    template_name = 'show_customer.html'

    def get_queryset(self):
        one_employee                                                =
Employee.objects.get(id=self.request.session['user_id'])
```

Продолжение листинга 3

```
        queryset = RequestFix.objects.filter(
            requester=one_employee,
            delete_date__isnull=True
        )
        search_query = self.request.GET.get('search', '')
        if search_query:
            queryset = queryset.filter(
                Q(problem_description__icontains=search_query) |

Q(equipment__inventory_number__icontains=search_query)
            ) return queryset
```

Функция create_request_customer позволяет сотруднику создать новую заявку на ремонт, ограничивая выбор оборудования только теми единицами, которые закреплены за кабинетом сотрудника, и автоматически присваивает начальный статус и дату регистрации. Код реализации представлен на листинге 4.

Листинг 4 – Create_request_customer

```
def create_request_customer(request):
    one_employee                                                =
Employee.objects.get(id=request.session['user_id'])
    equipment_list = Equipment.objects.filter(
        assigned_office=one_employee.office,
        delete_date__isnull=True
    )
    if request.method == 'POST':
        equipment = Equipment.objects.get(
            inventory_number=request.POST.get('equipment')
        )
        req = RequestFix(

problem_description=request.POST.get('problem_description'),
            registration_date=timezone.now(),
            requester=one_employee,
            equipment=equipment,
            repair_stage=RepairStage.objects.first())
        req.save()
        return redirect('show_customer')
    return render(request, 'create_request_customer.html')
```

Функция TechnicianRequestListView предоставляет технику полный доступ ко всем заявкам системы с расширенными возможностями фильтрации по заказчику, оборудованию и комментариям, а также реализует сортировку списка по дате, статусу или имени заказчика для удобного управления нагрузкой. Код реализации представлен на листинге 5.

Листинг 5 – TechnicianRequestListView

```
class TechnicianRequestListView(ListView):
    model = RequestFix
```

Продолжение листинга 5

```
    template_name = 'show_technician.html'

    def get_queryset(self):
        queryset = RequestFix.objects.all().select_related(
            'requester', 'assigned_technician', 'equipment'
        )
        search_query = self.request.GET.get('search', '')
        if search_query:
            queryset = queryset.filter(
                Q(problem_description__icontains=search_query) |
                Q(requester__last_name__icontains=search_query)
            )
        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['new_requests_count'] = RequestFix.objects.filter(
            assigned_technician__isnull=True
        ).count()
        return context
```

Функция show_new_requests выбирает только новые заявки, используя сложную фильтрацию через Q-объекты для выявления записей без исполнителя, даты выполнения или с начальным статусом. Данная функция реализована для того, чтобы техник сразу мог просмотреть созданные и еще не взятые на починку. Код представлен на листинге 6.

Листинг 6 – Show_new_requests

```
def show_new_requests(request):
    requests = RequestFix.objects.filter(
        Q(assigned_technician__isnull=True) |
        Q(completion_date__isnull=True)
    )
    return render(request, 'show_new_requests.html',
                  {'requests': requests})
```

Функция create_request_technician позволяет техническому специалисту создавать заявки от имени любого сотрудника с полным контролем над всеми полями, включая выбор исполнителя и дат. Код представлен на листинге 7.

Листинг 7 – Create_request_technician

```
def create_request_technician(request):
    if request.method == 'POST':
        customer                                                =
Employee.objects.get(id=request.POST.get('requester'))
        equipment = Equipment.objects.get(
            inventory_number=request.POST.get('equipment')
```

Продолжение листинга 7

```
        )
        req = RequestFix(

problem_description=request.POST.get('problem_description'),
            registration_date=timezone.now(),
            requester=customer,
            assigned_technician=Employee.objects.get(
                id=request.POST.get('assigned_technician')
            ),
            equipment=equipment
        )
        req.save()
        return redirect('show_technician')
    return render(request, 'create_request_technician.html')
```

Функция edit_request реализует механизм редактирования существующей заявки техником, позволяя изменять исполнителя, статус, комментарий и даты выполнения, а также добавлять запчасти и услуги с расчётом стоимости. Код представлен на листинге 8.

Листинг 8 – Edit_request

```
def edit_request(request, request_id):
    req = get_object_or_404(RequestFix, act_number=request_id)
    if request.method == 'POST':
        req.problem_description                                    =
request.POST.get('problem_description')
        req.assigned_technician = Employee.objects.get(
            id=request.POST.get('assigned_technician')
        )
        if request.POST.get('completion_date'):
            req.completion_date = timezone.make_aware(

datetime.strptime(request.POST.get('completion_date'),
                                '%Y-%m-%dT%H:%M')
            )
        req.save()
        return redirect('show_technician')
    return   render(request,   'edit_request.html',   {'request_obj':
req})
```

Функция delete_request_customer обеспечивает мягкое удаление заявки из системы по идентификатору, доступное только для авторизованного заказчика при условии выполнения заявки, при этом запись не удаляется из базы данных, а помечается датой удаления. Код представлен на листинге 9.

Листинг 9 – Delete_request_customer

```
def delete_request_customer(request, request_id):
    req = RequestFix.objects.get(act_number=request_id)
    if req.requester.id != request.session['user_id']:
        messages.error(request, 'Это не ваша заявка')
        return redirect('show_customer')
    if not req.completion_date:
        messages.error(request, 'Можно удалить только выполненную заявку')
        return redirect('show_customer')
    req.delete_date = timezone.now()
    req.save()
    return redirect('show_customer')
```

Функция delete_request_technician обеспечивает полное удаление заявки из системы по идентификатору, доступное только для авторизованных техников, при этом запись безвозвратно удаляется из базы данных. Код представлен на листинге 10.

Листинг 10 – Delete_request_technician

```
def delete_request_technician(request, request_id):
    req = RequestFix.objects.get(act_number=request_id)
    req.delete()
    messages.success(request, 'Заявка полностью удалена')
    return redirect('show_technician')
```

Функция statistics_view формирует отчётную информацию по заявкам, включая статистику по оборудованию, ежемесячное количество заявок и распределение по типам техники, используя агрегатные функции Django для группировки данных. Код представлен на листинге 11.

Листинг 11 – Statistics_view

```
def statistics_view(request):
    equipment_stats = RequestFix.objects.filter(
        delete_date__isnull=True
    ).values(
        'equipment__inventory_number',
        'equipment__model__name'
    ).annotate(
        breakdown_count=Count('act_number')
    ).order_by('-breakdown_count')[:10]

    total = RequestFix.objects.count()
    completed = RequestFix.objects.filter(
        completion_date__isnull=False
    ).count()

    return render(request, 'statistics.html', {
        'equipment_stats': list(equipment_stats),
```

Продолжение листинга 11

```
        'total_requests': total,
        'completed_requests': completed
    })
```

Функция EquipmentListView отображает список всего оборудования организации с возможностью поиска по инвентарному номеру или модели, фильтрации по типу и статусу, а также сортировки по различным параметрам. Код представлен на листинге 12.

Листинг 12 – EquipmentListView

```
class EquipmentListView(ListView):
    model = Equipment
    template_name = 'show_equipment.html'

    def get_queryset(self):
        queryset = Equipment.objects.filter(
            delete_date__isnull=True
        ).select_related(
            'model', 'type', 'status', 'assigned_office'
        )
        search_query = self.request.GET.get('search', '')
        if search_query:
            queryset = queryset.filter(
                Q(inventory_number__icontains=search_query) |
                Q(model__name__icontains=search_query)
            )
        return queryset
```

Функция edit_equipment позволяет техническому специалисту редактировать информацию об оборудовании, включая загрузку новой фотографии с автоматическим сохранением в директорию media и присвоением уникального имени файла. Код представлен на листинге 13.

Листинг 13 – Edit_equipment

```
def edit_equipment(request, inventory_number):
    equipment = get_object_or_404(Equipment,
                                  inventory_number=inventory_number)
    if request.method == 'POST':
        equipment.model_id = request.POST.get('model')
        equipment.type_id = request.POST.get('type')
        photo_file = request.FILES.get('photo')
        if photo_file:
            filename                                                =
f"equipment_{equipment.inventory_number}_{timezone.now()}.jpg"
            photo = Photos.objects.create(name=filename)
            equipment.photo = photo
        equipment.save()
        return redirect('show_equipment')
```

Продолжение листинга 13

```
    return render(request, 'edit_equipment.html')
```

Функция equipment_costs рассчитывает общую стоимость обслуживания оборудования за выбранный период, суммируя затраты на запчасти и услуги по всем связанным заявкам, а также вычисляет среднюю стоимость одной заявки. Код представлен на листинге 14.

Листинг 14 – Equipment_costs

```
def equipment_costs(request):
    equipment_id = request.GET.get('equipment_id')
    requests = RequestFix.objects.filter(
        equipment_id=equipment_id,
        delete_date__isnull=True
    ).prefetch_related('used_spare_parts', 'used_services')

    total_cost = 0
    for req in requests:
        spare_parts_cost = sum(item.total_cost
                               for              item              in
req.used_spare_parts.all())
        services_cost = sum(item.total_cost
                            for item in req.used_services.all())
        total_cost += spare_parts_cost + services_cost

    return render(request, 'equipment_costs.html', {
        'total_cost': total_cost,
        'requests': requests
    })
```

Функция index обеспечивает перенаправление пользователя на соответствующую страницу в зависимости от наличия активной сессии и роли, осуществляя вход в систему без дополнительной авторизации при сохранённых данных сессии. Код представлен на листинге 15.

Листинг 15 – Index

```
def index(request):
    if 'user_id' in request.session:
        if request.session.get('user_role') == 'Техник':
            return redirect('show_technician')
        return redirect('show_customer')
    return redirect('login_view')
```

5 ТЕСТИРОВАНИЕ ПРОГРАММНОГО МОДУЛЯ

5.1 Разработка тест-кейсов

Этап тестирования является неотъемлемой частью жизненного цикла разработки программного обеспечения, позволяющей убедиться в соответствии реализованного функционала первоначальным требованиям и спецификации. В рамках данного раздела были разработаны тест-кейсы, описывающие последовательность действий пользователя и ожидаемые результаты работы системы для различных сценариев использования. Посмотреть тест-кейсы можно в таблицах 1-5.

Таблица 1 – Тест-кейс №1

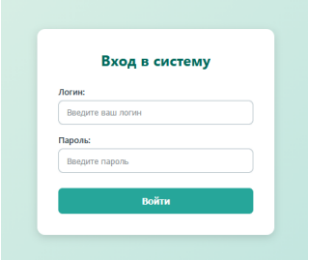
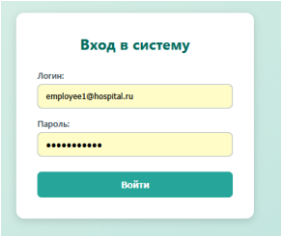
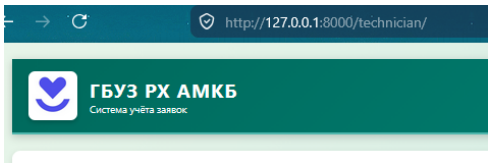
Номер	1
Название	Тестирование входа в аккаунт с ролью «Техник»
Предусловие	Запущен сервер разработки, база данных заполнена тестовыми данными
Шаг №1	Запустить программу через python manage.py runserver и перейти на страницу http://127.0.0.1:8000/
Результат шага №1	Отображение окна с наименованием «Вход в систему», поля для ввода с наименованиями «Логин» и «Пароль», кнопка «Войти» 
Шаг №2	Ввести логин employee1@hospital.ru и пароль password123 с существующими данными в таблице, где пароль и логин будут соответствовать друг другу, нажать на кнопку «Войти» 
Результат шага №2	Перенаправление на страницу техника http://127.0.0.1:8000/technician/ 
Поведение	Позитивное
Статус	Пройден

Таблица2 – Тест-кейс №2

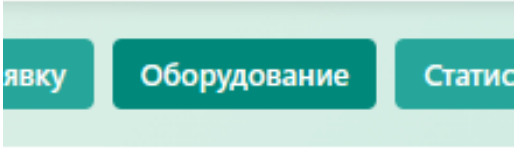
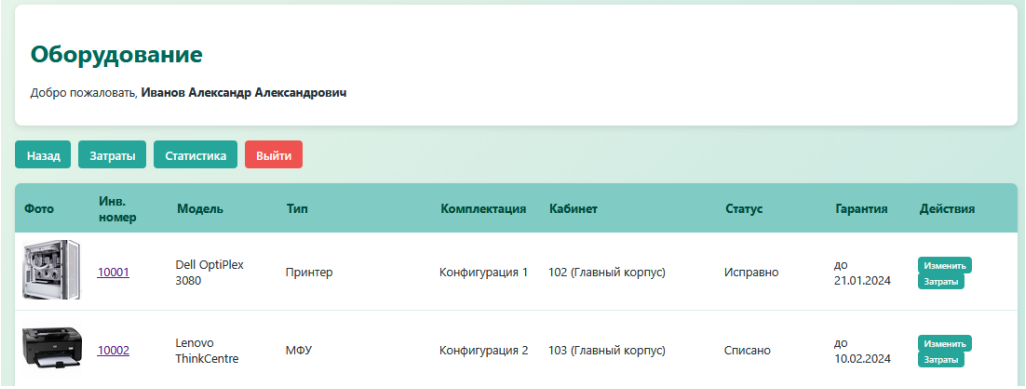
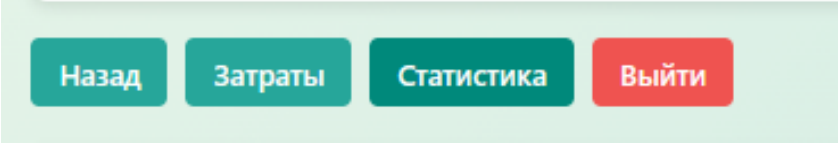
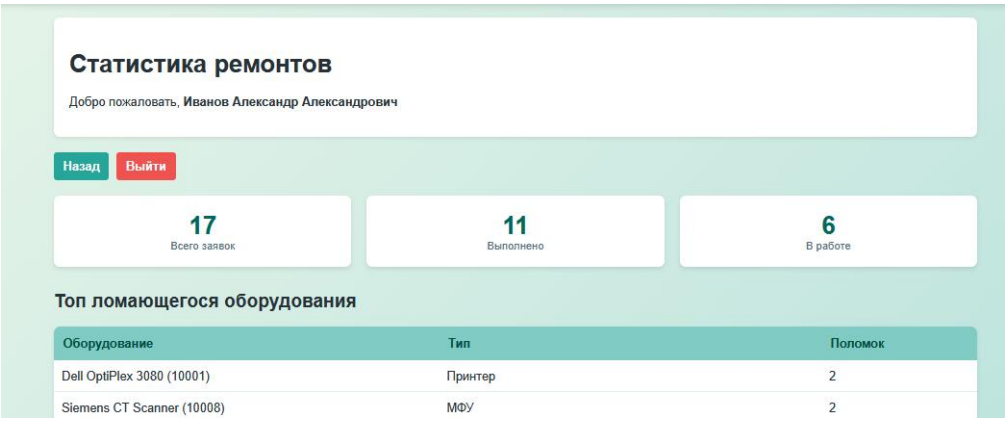
Номер	2
Название	Тестирование навигации из техника на страницу оборудования
Предусловие	Уже вошли под сотрудником с ролью «Техник»
Шаг №1	Кликнуть по кнопке на странице «Оборудование» 
Результат шага №1	Отображение страницы оборудования со списком всех единиц техники, закреплённых за кабинетами организации 
Шаг №2	Кликнуть по кнопке на странице «Статистика» 
Результат шага №2	Отображение страницы статистики с данными о количестве заявок, выполненных работах и типах оборудования 
Поведение	Позитивное
Статус	Пройден

Таблица 3 – Тест-кейс №3

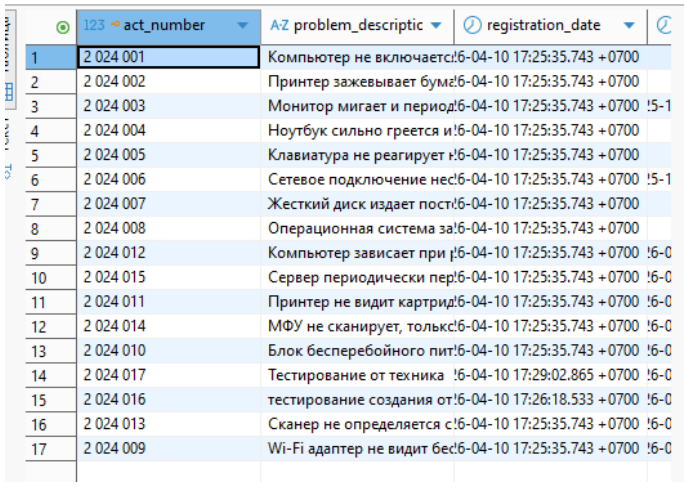
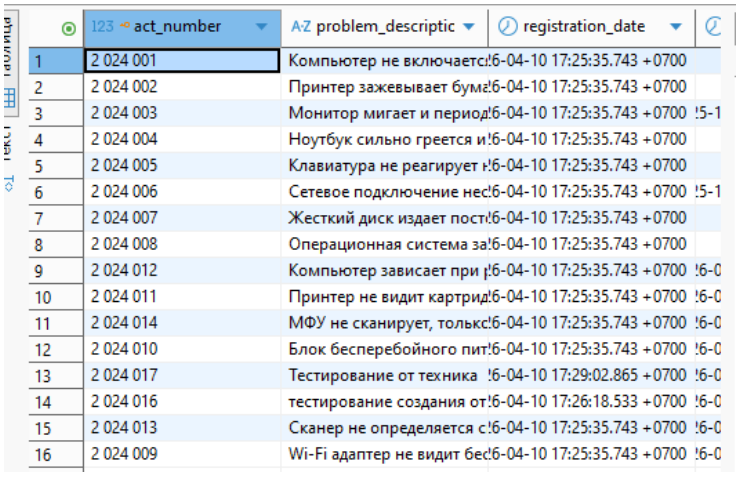
Номер	3
Название	Тестирование удаления заявки
Предусловие	Уже вошли под сотрудником с ролью «Техник», в системе существует минимум одна заявка
Шаг №1	На странице техника нажать на кнопку «Удалить» рядом с заявкой  act_number / problem_descriptic / registration_date 1 2 024 001 Компьютер не включаетс 6-04-10 17:25:35.743 +0700 2 2 024 002 Принтер зажевывает бум 6-04-10 17:25:35.743 +0700 3 2 024 003 Монитор мигает и период 6-04-10 17:25:35.743 +0700 5-1 4 2 024 004 Ноутбук сильно греется и 6-04-10 17:25:35.743 +0700 5 2 024 005 Клавиатура не реагирует н 6-04-10 17:25:35.743 +0700 6 2 024 006 Сетевое подключение нес 6-04-10 17:25:35.743 +0700 5-1 7 2 024 007 Жесткий диск издает пост 6-04-10 17:25:35.743 +0700 8 2 024 008 Операционная система за 6-04-10 17:25:35.743 +0700 9 2 024 012 Компьютер зависает при р 6-04-10 17:25:35.743 +0700 6-0 10 2 024 015 Сервер периодически пер 6-04-10 17:25:35.743 +0700 6-0 11 2 024 011 Принтер не видит картрид 6-04-10 17:25:35.743 +0700 6-0 12 2 024 014 МФУ не сканирует, толькс 6-04-10 17:25:35.743 +0700 6-0 13 2 024 010 Блок бесперебойного пит 6-04-10 17:25:35.743 +0700 6-0 14 2 024 017 Тестирование от техника 6-04-10 17:29:02.865 +0700 6-0 15 2 024 016 тестирование создания от 6-04-10 17:26:18.533 +0700 6-0 16 2 024 013 Сканер не определяется с 6-04-10 17:25:35.743 +0700 6-0 17 2 024 009 Wi-Fi адаптер не видит бес 6-04-10 17:25:35.743 +0700 6-0
Результат шага №1	Отправка POST-запроса на удаление, заявка удаляется из базы данных
Шаг №2	Проверить количество заявок в системе
Результат шага №2	Количество заявок уменьшилось на 1, заявка с указанным номером акта больше не существует в базе данных и на странице  act_number / problem_descriptic / registration_date 1 2 024 001 Компьютер не включаетс 6-04-10 17:25:35.743 +0700 2 2 024 002 Принтер зажевывает бум 6-04-10 17:25:35.743 +0700 3 2 024 003 Монитор мигает и период 6-04-10 17:25:35.743 +0700 5-1 4 2 024 004 Ноутбук сильно греется и 6-04-10 17:25:35.743 +0700 5 2 024 005 Клавиатура не реагирует н 6-04-10 17:25:35.743 +0700 6 2 024 006 Сетевое подключение нес 6-04-10 17:25:35.743 +0700 5-1 7 2 024 007 Жесткий диск издает пост 6-04-10 17:25:35.743 +0700 8 2 024 008 Операционная система за 6-04-10 17:25:35.743 +0700 9 2 024 012 Компьютер зависает при р 6-04-10 17:25:35.743 +0700 6-0 10 2 024 011 Принтер не видит картрид 6-04-10 17:25:35.743 +0700 6-0 11 2 024 014 МФУ не сканирует, толькс 6-04-10 17:25:35.743 +0700 6-0 12 2 024 010 Блок бесперебойного пит 6-04-10 17:25:35.743 +0700 6-0 13 2 024 017 Тестирование от техника 6-04-10 17:29:02.865 +0700 6-0 14 2 024 016 тестирование создания от 6-04-10 17:26:18.533 +0700 6-0 15 2 024 013 Сканер не определяется с 6-04-10 17:25:35.743 +0700 6-0 16 2 024 009 Wi-Fi адаптер не видит бес 6-04-10 17:25:35.743 +0700 6-0
Шаг №3	Попытаться удалить невыполненную заявку от имени сотрудника
Результат шага №3	Отображение сообщения «Можно удалить только выполненную заявку», заявка остаётся в системе
Поведение	Негативное
Статус	Пройден

Таблица 4 – Тест-кейс №4

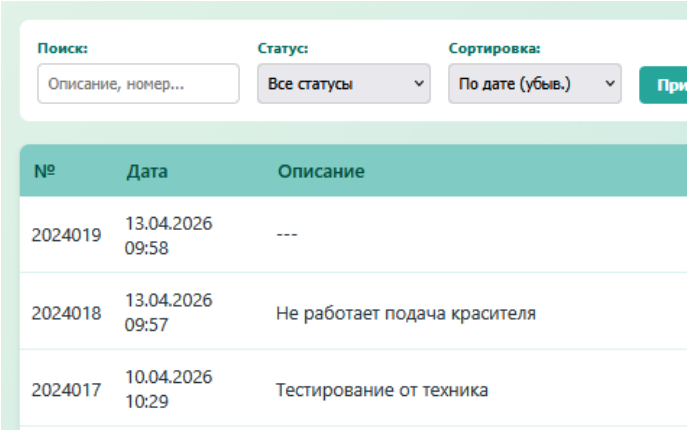
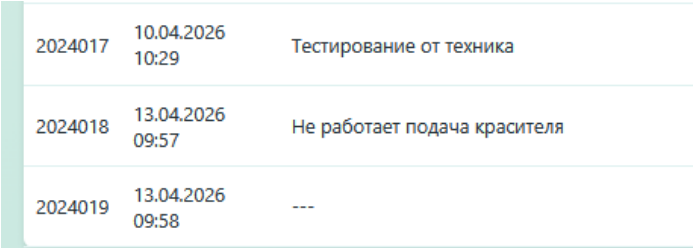
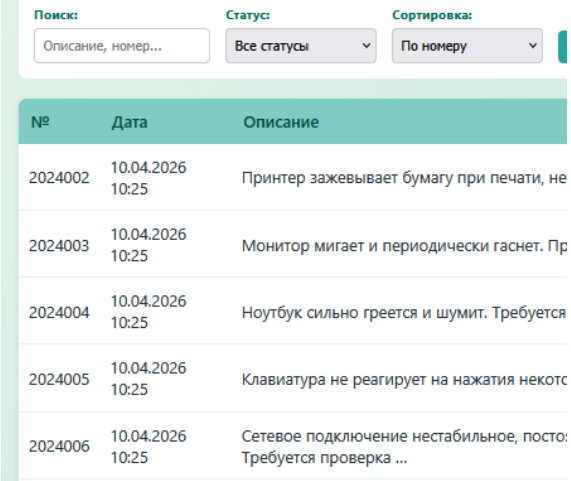
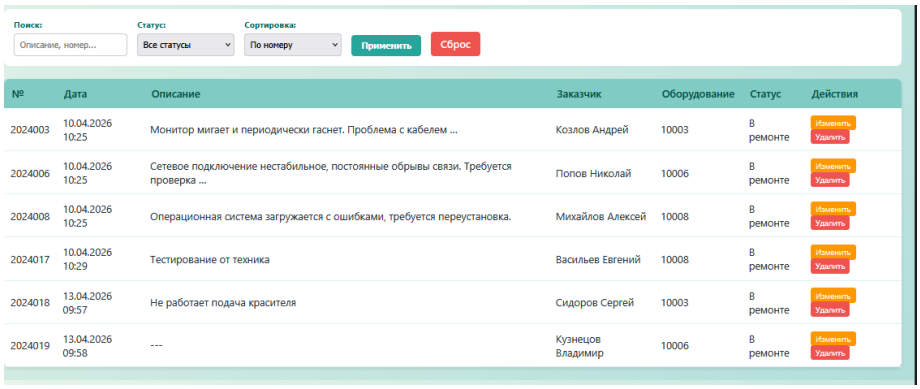
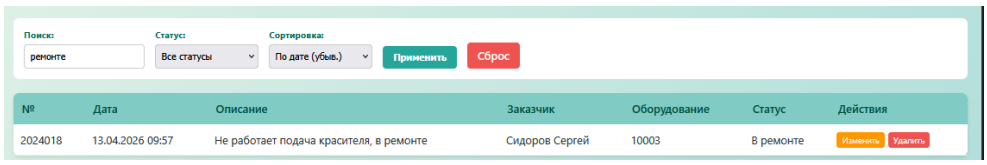
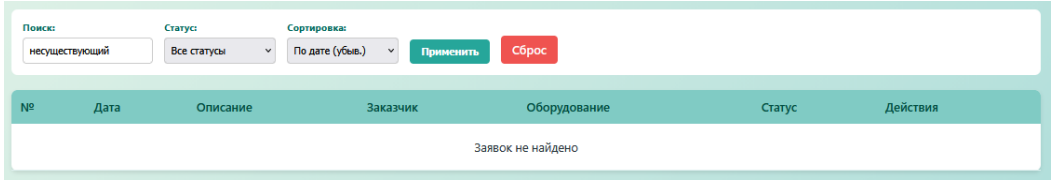
Номер	4
Название	Тестирование сортировки по убыванию на странице техника
Предусловие	Уже вошли под сотрудником с ролью «Техник», в системе существует минимум 3 заявки с разными датами регистрации
Шаг №1	Кликнуть по кнопке на странице «По дате (убыв.)»
Результат шага №1	Отображение заявок от последней к самой первой 
Шаг №2	Кликнуть по кнопке «По дате (возр.)»
Результат шага №2	Отображение заявок от первой к последней 
Шаг №3	Кликнуть по кнопке «По номеру»
Результат шага №3	Отображение заявок отсортированных по номеру акта в порядке возрастания 
Поведение	Позитивное
Статус	Пройден

Таблица 5 – Тест-кейс №5

Номер	5
Название	Тестирование фильтрации по статусу
Предусловие	Уже вошли под сотрудником с ролью «Техник», в системе существуют заявки с разными статусами оборудования
Шаг №1	Выбрать в поле «Статус» один из статусов, к примеру «В ремонте»
Результат шага №1	Отображение заявок со статусом «В ремонте» 
Шаг №2	Ввести в поле «Поиск» текст «ремонте» и нажать «Применить»
Результат шага №2	Отображение заявок, в описании которых встречается слово «ремонте», остальные заявки скрыты 
Шаг №3	Ввести в поле «Поиск» текст «несуществующий текст»
Результат шага №3	Отображение сообщения «Заявок не найдено», список заявок пуст 
Поведение	Негативное
Статус	Пройден

5.2 Модульное тестирование

Проведение модульного тестирования необходимо для верификации корректности работы отдельных компонентов программного средства и выявления ошибок логики на ранних этапах. Основной целью данного этапа является подтверждение того, что функции приложения выполняют заложенную логику без сбоев.

Для автоматизации процесса тестирования использовались встроенные средства фреймворка Django на языке программирования Python, соответствующие выбранным системным характеристикам проекта. В качестве инструментальной базы применялись классы TestCase и клиент тестирования Django, позволяющие эмулировать HTTP-запросы и проверять ответы сервера, статусы моделей базы данных и корректность перенаправлений.

Такой подход позволил подтвердить работоспособность критических узлов системы, включая механизмы аутентификации, применения фильтров и сортировки, обеспечивая надежность и стабильность разработанного программного модуля перед его внедрением.

Посмотреть листинги тестирования можно в соответствии с листингами 16-20.

Листинг 16 – Тестирование входа на сайт через логин и пароль

```
class TestLogin(TestCase):
    def setUp(self):
        self.role = Role.objects.create(id=1, role_name='Техник')
        self.employee = Employee.objects.create(
            id=1, login='tech1@hospital.ru', password='password123',
            last_name='Иванов', first_name='Иван', role_id=1
        )
        self.client = Client()

    def test_login_success(self):
        response = self.client.post(reverse('login_view'), {
            'username':       'tech1@hospital.ru',       'password':
'password123'
        })
        self.assertEqual(response.status_code, 302)
        self.assertEqual(self.client.session['user_id'], 1)
        self.assertRedirects(response, reverse('show_technician'))
```

Листинг 17 – Тестирование кнопок навигации

```
class TestNavigation(TestCase):
    def setUp(self):
        self.role = Role.objects.create(id=1, role_name='Техник')
        self.employee = Employee.objects.create(
            id=1, login='tech1@hospital.ru', password='password123',
            last_name='Иванов', first_name='Иван', role_id=1
        )
        self.client = Client()
        self.client.post(reverse('login_view'), {
            'username':       'tech1@hospital.ru',       'password':
'password123'
        })

    def test_navigation_to_equipment(self):
        response = self.client.get(reverse('show_equipment'))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Оборудование')
```

Листинг 18 – Тестирование кнопки удаления

```
class TestDeleteRequest(TestCase):
    def setUp(self):
        self.role = Role.objects.create(id=1, role_name='Техник')
        self.employee = Employee.objects.create(
            id=1, login='tech1@hospital.ru', password='password123',
            last_name='Иванов', first_name='Иван', role_id=1
        )
        self.request = RequestFix.objects.create(
            act_number=2024001, problem_description='Тест',
            requester_id=1, equipment_id=10001,
            completion_date=timezone.now()
        )
        self.client = Client()
        self.client.login(username='tech1@hospital.ru',
password='password123')

    def test_delete_request(self):
        self.assertEqual(RequestFix.objects.count(), 1)
        response                                                =
self.client.post(reverse('delete_request_technician',
args=[2024001]))
        self.assertEqual(response.status_code, 302)
        self.assertEqual(RequestFix.objects.count(), 0)
```

Листинг 19 – Тестирование сортировки

```
class TestSorting(TestCase):
    def setUp(self):
        self.role = Role.objects.create(id=1, role_name='Техник')
        self.employee        =        Employee.objects.create(id=1,
login='tech1@hospital.ru',
password='password123',last_name='Иванов',        first_name='Иван',
role_id=1)
        RequestFix.objects.create(act_number=2024001,
problem_description='Тест   1',requester_id=1,   equipment_id=10001,
registration_date='2024-01-01')
        RequestFix.objects.create(act_number=2024002,
problem_description='Тест 2',
            requester_id=1,                     equipment_id=10001,
registration_date='2024-01-02')
        self.client = Client()
        self.client.login(username='tech1@hospital.ru',
password='password123')
    def test_sort_by_date_desc(self):
        response    =    self.client.get(reverse('show_technician'),
{'sort': '-registration_date'})
        self.assertEqual(response.status_code, 200)
        requests = response.context['requests']
```

Продолжение листинга 19

```
            self.assertEqual(requests[0].act_number, 2024002)
            self.assertEqual(requests[1].act_number, 2024001)
```

Листинг 20 – Тестирование фильтрации

```
class TestFiltering(TestCase):
    def setUp(self):
        self.role = Role.objects.create(id=1, role_name='Техник')
        self.employee = Employee.objects.create(
            id=1, login='tech1@hospital.ru', password='password123',
            last_name='Иванов', first_name='Иван', role_id=1
        )
        self.status = EquipmentStatus.objects.create(id=1, name='В
ремонте')
        self.equipment = Equipment.objects.create(
            inventory_number=10001, status_id=1
        )
        RequestFix.objects.create(act_number=2024001,
problem_description='Проблема с мышью',
            requester_id=1, equipment_id=10001)
        RequestFix.objects.create(act_number=2024002,
problem_description='Проблема с клавиатурой',
            requester_id=1, equipment_id=10001)
        self.client = Client()
        self.client.login(username='tech1@hospital.ru',
password='password123')

    def test_filter_by_status(self):
        response = self.client.get(reverse('show_technician'),
{'status': '1'})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'Проблема с мышью')
```

Посмотреть выполнение всего файла с тестами можно в соответствии с рисунком 5.1.

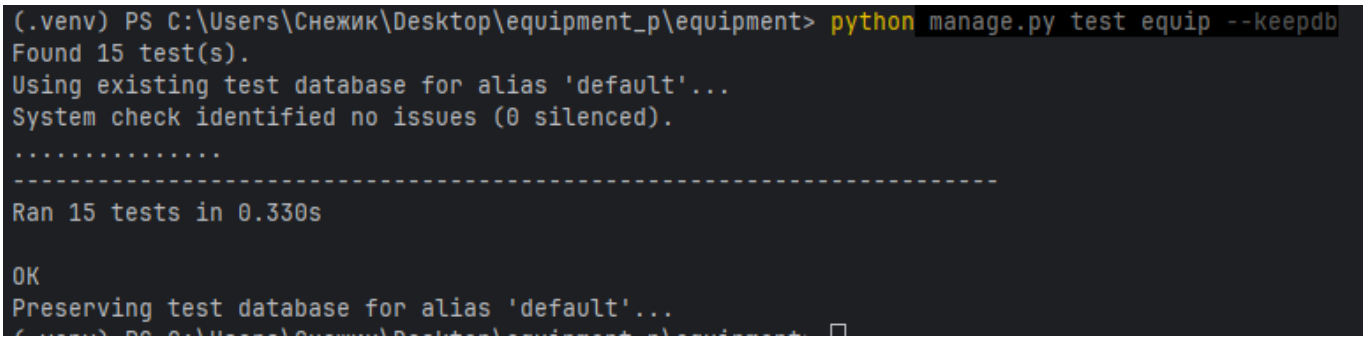


Рисунок 5.1 – Сообщение о корректном выполнении всего файла с тестами программы

ПРИЛОЖЕНИЯ

Спецификация требований

Требования к роли «Сотрудник»:

- Авторизация в системе по логину и паролю;
- Просмотр списка своих заявок;
- Создание новой заявки на ремонт;
- Выбор оборудования из кабинета сотрудника;
- Указание описания неисправности;
- Выбор категории и приоритета заявки;
- Просмотр статуса своих заявок;
- Поиск и фильтрация своих заявок;
- Удаление только выполненных заявок;
- Просмотр исполнителя заявки;
- Переключение в режим техника (если есть права);
- Выход из системы.

Требования к роли «Техник»:

- Авторизация в системе по логину и паролю;
- Просмотр всех заявок организации;
- Просмотр новых заявок;
- Создание заявки от имени сотрудника;
- Редактирование любой заявки;
- Назначение исполнителя на заявку;
- Изменение статуса заявки и этапа ремонта;
- Указание даты и времени выполнения;
- Добавление комментария техника;
- Полное удаление любой заявки;
- Поиск, фильтрация, сортировка заявок;
- Просмотр списка оборудования;
- Просмотр затрат по оборудованию;
- Просмотр статистики ремонтов;
- Переключение в режим сотрудника;
- Выход из системы.

Функциональные требования:

- Вход в систему с проверкой логина и пароля из БД;
- Автоматическое определение роли пользователя;
- Хранение данных пользователя в session;
- Очистка сессии при выходе;
- Проверка прав доступа к страницам;
- Создание заявки с обязательными полями
- Автофиксация даты регистрации;
- Автогенерация номера заявки;
- Редактирование всех полей заявки;
- Мягкое удаление для сотрудника;
- Полное удаление для техника;
- Валидация обязательных полей;
- Блокировка повторного открытия формы через session;
- Поиск по описанию, номеру, ФИО, оборудованию;
- Фильтрация по статусу оборудования;
- Фильтрация по сотруднику (заказчику);
- Сортировка по дате (возрастание/убывание);
- Сортировка по номеру заявки;
- Комбинирование поиска + фильтр;
- Просмотр списка всего оборудования;
- Фильтрация по типу оборудования;
- Фильтрация по статусу оборудования;
- Поиск по инвентарному номеру и модели;
- Просмотр статистики по ремонтам оборудования;
- История всех заявок с данным оборудованием;
- Расчет общей и средней стоимости ремонта;
- Просмотр списка всего оборудования;
- Фильтрация по типу оборудования;
- Фильтрация по статусу оборудования;
- Поиск по инвентарному номеру и модели;
- Просмотр статистики по ремонтам оборудования;
- История всех заявок с данным оборудованием;

— Расчет общей и средней стоимости ремонта;

— Всплывающие уведомления;

— Обработка ошибок через try/except;

— Валидация форм с сообщениями об ошибках

— Подтверждение перед удалением;

Нефункциональные требования:

— Обязательная авторизация для всех страниц;

— Проверка роли перед доступом к функциям;

— Хранение user_id в session;

— CSRF защита во всех формах;

— Мягкое удаление для сотрудников.

Детальные спецификации по каждым таблицам главных объектов представлены на таблицах 6-8.

Таблица 6 - Спецификация объекта «Заявка»

Поле	Тип	Обязательно	Описание
act_number	INTEGER (PK	Да	Уникальный номер заявки
requester	INTEGER (FK)	Да	Сотрудник, создавший заявку
assigned_technician	INTEGER (FK)	Нет	Техник, назначенный на заявку
equipment	INTEGER (FK)	Да	Оборудование, требующее ремонта
category	INTEGER (FK)	Да	Категория неисправности
repair_stage	INTEGER (FK)	Да	Текущий этап ремонта
priority	INTEGER (FK)	Да	Приоритет заявки
problem_description	VARCHAR(500)	Да	Текстовое описание проблемы/неисправности оборудования
registration_date	DATETIME	Да	Дата и время создания заявки
completion_date	DATETIME	Нет	Дата и время завершения ремонта
spare_parts	MTM (FK)	Да	Связь многие-ко-многим с таблицей запчастей

Таблица 7 - Спецификация объекта «Сотрудник»

Поле	Тип	Обязательно	Описание
id	VARCHAR(250)	Да	Первичный ключ
login	VARCHAR(250)	Да	Почта сотрудника
last_name	VARCHAR(250)	Да	Фамилия сотрудника
first_name	VARCHAR(250)	Да	Имя сотрудника
middle_name	VARCHAR(250)	Нет	Отчество сотрудника
phone_number	VARCHAR(20)	Нет	Контактный телефон
position	INTEGER (FK)	Да	Ссылка на должность
department	INTEGER (FK)	Да	Ссылка на отдел
last_login	DATETIME	Нет	Ссылка время последнего входа
password	VARCHAR(250)	Да	Пароль для входа
role	INTEGER (FK)	Да	Ссылка на таблицу ролей, роль в системе
office	INTEGER (FK)	Да	Ссылка на кабинет

Таблица 8 - Спецификация объекта «Оборудование»

Поле	Тип	Обязательно	Описание
inventory_number	INTEGER (FK)	Да	Первичный ключ
assigned_office	INTEGER (FK)	Да	Ссылка на кабинет
type	INTEGER (FK)	Да	Ссылка на тип оборудования
configuration	VARCHAR(250)	Да	Описание комплекции
model	INTEGER (FK)	Да	Ссылка на модель
status	INTEGER (FK)	Да	Ссылка на статус
warranty	INTEGER (FK)	Нет	Ссылка на гарантию
purchase_date	DATE	Да	Дата приобретения оборудования организацией
photo	INTEGER (FK)	Да	Фотография оборудования

Так же были составлены бизнес-правила. Они представляют собой формализованные ограничения и условия, определяющие логику функционирования программного средства в рамках предметной области. Наличие чётко сформулированных бизнес-правил обеспечивает согласованность работы всех компонентов системы, предотвращает возникновение противоречий в обработке данных и гарантирует соответствие результатов деятельности программного модуля требованиям организации.

Реализация бизнес-правил на уровне приложения позволяет автоматизировать контроль за соблюдением регламентов, минимизировать влияние человеческого фактора и обеспечить целостность хранимой информации. Бизнес правила для системы представлены на таблице 9.

Таблица 9 - Бизнес правила

Правило	Описание
Обязательные поля заявки	Заявка не создаётся без: ФИО заказчика, инвентарного номера, описания неисправности
Уникальность инвентарного номера	Инвентарный номер оборудования уникален в системе
Назначение исполнителя	На одну заявку — только один исполнитель
Срок выполнения	Заявка выполняется в течение нужного количества рабочих дней с регистрации в зависимости от типа поломки и типа оборудования
Доступ к данным	Пользователь видит только заявки, связанные с ним в отношении заказчика
Редактирование заявки	Редактирование возможно только техником
Комментарий при закрытии	Исполнитель может добавлять свои комментарий о работе

Репозиторий с документами в системе контроля версий

GIT - https://github.com/SnezhiK000/Practica_Equipment